

CNN-based Hardware Design for Real-time EMG-based Hand Gesture Recognition at the Edge

A THESIS SUBMITTED
IN PARTIAL FULFILMENT OF THE
REQUIREMENTS FOR THE DEGREE OF
Master of Technology
IN THE
Faculty of Engineering

BY

Pradhuman Singh(22747),Swarnima(22963)

SUPERVISED BY

Dr. Debayan Das



Department of Electrical and Communication Engineering
Indian Institute of Science
Bengaluru 560 012 (INDIA)

June 2025

Abstract

This paper presents a low-power convolutional neural network (CNN)-based hand gesture recognition (HGR) system using real-time electromyography (EMG) signals, targeting wearable edge devices. EMG-based gesture recognition enables intuitive interaction in augmented and virtual reality (AR/VR) environments; however, achieving real-time performance with low power consumption on resource-constrained edge devices remains a key challenge. To address this, we propose an energy-efficient hardware implementation of a 1D CNN-based model using 1D EMG signal (1×6) input (Model 1). The EMG data is first preprocessed and temporally aligned using Dynamic Time Warping (DTW) to improve consistency across gesture samples. We compare two CNN architectures: Model 1 processes 1D EMG inputs directly, while Model 2 uses the same 1D CNN architecture but on 2D-transformed EMG input with dimension 1019×6 . Our optimized hardware design for Model 1 leverages intelligent hardware reuse, a lightweight input interface, and handshake techniques to achieve high efficiency. Implemented on an FPGA, Model 1 achieves maximum frequency of 55.2 MHz with a inference time of $8.1 \mu s$ at 50 MHz, consuming only 0.022 W of power. In contrast, Model 2, which uses 2D inputs, has a maximum frequency of 75.9 MHz, but exhibits significantly higher latency of $84.72 \mu s$ at 50 Mhz and power consumption of 0.519 W. These results demonstrate that using a 1D input representation combined with a tailored hardware accelerator can significantly reduce energy and time costs, making our solution highly suitable for real-time, on-device EMG-based HGR in wearable applications.

Acknowledgements

We would like to express our deepest gratitude to all those who have supported us throughout our journey in this project and the process of writing this thesis. Without the guidance, encouragement, and assistance of many, this work would not have been possible. We are especially thankful to our advisor, Dr. Debayan Das, head of the Circuits and Security Research (CSR) Lab, Department of Electronic Systems Engineering (DESE), Indian Institute of Science (IISc), for his constant support, insightful guidance, and invaluable expertise throughout this journey. His encouragement and dedication have played a pivotal role in the successful completion of this thesis. We would also like to express our sincere gratitude to Madhuri, M.Tech AI, our friend and colleague, whose assistance with setting up the software support was invaluable to the success of this project. Our deep appreciation goes to the members of the CSRL Lab, Department of Electronic Systems Engineering (DESE), IISc, for fostering an environment that was both intellectually stimulating and incredibly supportive. We are truly grateful to our friends and classmates from the Microelectronics and VLSI Design 2023-25 batch for their constant moral support and encouragement throughout our research and development efforts. Finally, we would like to express our deep appreciation for our institution, IISc, for providing us with the resources, facilities, and a welcoming community that have been vital for the successful execution of our project.

Notation and Abbreviations

List of Symbols

List of Abbreviations

HGR	Hand Gesture Recognition System
1D CNN	1 Dimensional Convolutional Neural Network
EMG	Electromyography
DTW	Dynamic Time Warping
PCA	Principal component Analysis
Conv1D	Convolution 1 Dimensional
FPGA	Field Programmable Gate Array
IMU	Inertial Measurement Unit
DQN	Deep Q-Network
RNN	Recurrent Neural Network
sEMG	Surface Electromyography
GPUs	Graphics Processing Units
CPUs	Central Processing Units
PCBs	Printed Circuit Boards
HW	Hardware
AR/VR	Augmented Reality / Virtual Reality
ML	Machine Learning
IoT	Internet of Things
BRAM	Block Random Access Memory
ASIC	Application-Specific Integrated Circuit
SIPO	Serial-In Parallel-Out
MUX	Multiplexer
GAP	Global average pooling

Contents

Abstract	i
Acknowledgements	ii
Notation and Abbreviations	iii
List of Figures	vii
List of Tables	ix
1 Introduction	1
1.1 1D Convolution	2
1.1.1 1D Convolution with 1D Input	2
1.1.2 1D Convolution with 2D Input	4
1.2 Max Pooling	6
1.2.1 1D Max Pooling	6
1.2.2 2D Max Pooling	6
1.3 Dense Layer	7
1.3.1 Importance of Dense Layers	8
1.4 Activation Functions	9
1.4.1 ReLU Activation Function	9
1.4.2 Tanh Activation Function	9
2 Motivation	11
3 Background Study and Related works	13
3.1 Reinforcement Learning-Based Hand Gesture Recognition Systems Using EMG and IMU Signals [1]	13

3.2	Neurobehavioral Development of Emotion-Specific Spontaneous Facial Responses in Infants [2]	15
3.3	EMG Analysis of Tibialis Anterior and Gastrocnemius in ASD Gait [3] .	17
3.4	FPGA-Based Acceleration of CNNs for Real-Time Digit Recognition [4] .	18
3.5	Portable Myoelectric Control Systems Using Quantized CNNs on FPGA Hardware [5]	19
4	EMG Classifier design	22
4.1	Proposed Approach 1	22
4.1.1	Model 1: (Single Channel)	22
4.1.2	CNN Architecture Model 1	23
4.1.3	Dataset	26
4.2	Limitations of Single-Channel Input and Motivation for Multi-Channel Architecture	27
4.3	Proposed Approach 2	27
4.3.1	Model 2:(Multi channel)	27
4.3.2	CNN Architecture Model 2	28
5	Hardware Implementation	31
5.1	Implementation of Model 1	31
5.1.1	Architecture of Model 1 CNN	31
5.1.2	Architecture of conv1D Layer1(Model1)	36
5.1.3	Architecture of conv1D Layer 2(Model 1)	38
5.1.4	Architecture of dense layer	40
5.2	Implementation of Model 2	43
5.2.1	Architecture of Model 2 CNN	43
5.2.2	Architecture of conv1D layer 1(Model 2)	49
5.2.3	Architecture of maxpool layer(Model 2)	53
5.2.4	Architecture of conv1D layer2(Model 2)	56
5.2.5	Architecture of Global Average Pooling (GAP)	61
6	Results	68
6.1	Simulation output of CNN Model 1	68
6.2	Simulation outputs of CNN Model 2	71

6.3	Comparison between Resource utilisation	73
6.4	Comparison between Performamnce metrics	75
7	Conclusion	76
8	Future work	78
A	Results	80
B	Code Repository	82
	Bibliography	83

List of Figures

1-1	Hand Gesture Recognition System	2
3-1	HGR system utilizing EMG-IMU signals and deep neural network architecture for gesture classification and robotic control [1]	13
3-2	CNN Model used in digit recognition	18
3-3	Hardware architecture used in this work [5]	19
4-1	Neural Network Architecture used in this approach	25
4-2	Confusion Matrices: Test Accuracy of the CNN Model 1(single channel) using (a) Floating-point parameters and (b) Fixed-point (Integer) parameters.	26
4-3	Confusion Matrices: Test Accuracy of the CNN Model 2(six channels) using (a) Floating-point parameters and (b) Fixed-point (Integer) parameters.	30
5-1	Block diagram of Architecture of Model 1	31
5-2	Block diagram of architecture of conv1D layer1(Model1).	36
5-3	Block diagram of architecture of conv1D layer2(Model1).	38
5-4	Block diagram of architecture of dense layer.	40
5-5	Block diagram of Architecture of model 2	43
5-6	Block diagram of Architecture of conv1D layer1(Model 2)	49
5-7	Architecture of maxpool unit, pool size 2	53
5-8	Block diagram of Architecture of layer 2(Model2)	56
5-9	Block diagram of Architecture of global average pooling	61
6-1	Simulation result for conv 1D	68
6-2	Simulation result for ReLU	68
6-3	Simulation result for MaxPool	69
6-4	Simulation result of conv1D Layer 1	69

6-5	Simulation result for Dense Layer 1	70
6-6	Simulation result of Dense Layer 2	70
6-7	Simulation result of complete Model 1	71
6-8	Simulation result for Conv1D layer in Model2.	71
6-9	Simulation Result of CNN Model 2	72
A-1	Post Implementation Timing Report(Model 2)	80
A-2	Post Implementation Power report(Model 2)	80
A-3	Post implementation Resource utilization perentage(Model 2)	81
A-4	Post implementation Resource utilization of individual module(Model 2)	81

List of Tables

2.1	Comparison of FPGA, CPU, and GPU for Hand Gesture Recognition at the Edge	12
5.1	Summary of CNN Architecture and data flow in Model 1	42
5.2	Summary of CNN Model 2 Architecture and Data Flow	67
6.1	Resource Utilization	73
6.2	Performance Metrics	75

Chapter 1

Introduction

The cutting-edge gesture recognition system, as depicted in fig.1-1 integrates machine learning with hardware acceleration. The system utilizes a convolutional neural network (CNN), a powerful machine learning model, to analyze electromagnetic (EMG) signals captured from sensors. These signals, representing the electrical activity of muscles during hand movements, are pre-processed and then fed into the CNN. CNN, implemented on a Field-Programmable Gate Array (FPGA), leverages the FPGA's parallel processing capabilities to perform high-speed real-time analysis of EMG data. This enables the system to accurately and efficiently recognize a variety of hand gestures, opening up a wide range of applications in fields such as gaming, human-computer interaction, and assistive technologies

An FPGA, a highly programmable hardware device with notable benefits in processing speed, power efficiency, and real-time performance, is used to create this CNN. In order to find patterns and characteristics typical of various hand motions, such as an open hand, pinch, or fist, the CNN examines the processed EMG signals. Following recognition, the gestures are exported for a variety of uses, such as robotics, assistive technology, gaming, and human-computer interface. The training phase and the inference phase are the two stages of a CNN model. A CNN model consists of two phases: the training phase and the inference phase. In the training phase, the model learns the network's parameters by performing forward and backward propagations. Once the network is trained, it can be used for inference utilizing the required weights and biases (learned parameters), which can be used to perform forward propagation to make predictions. The inference phase does not need as much computing power as the training phase and takes much less time

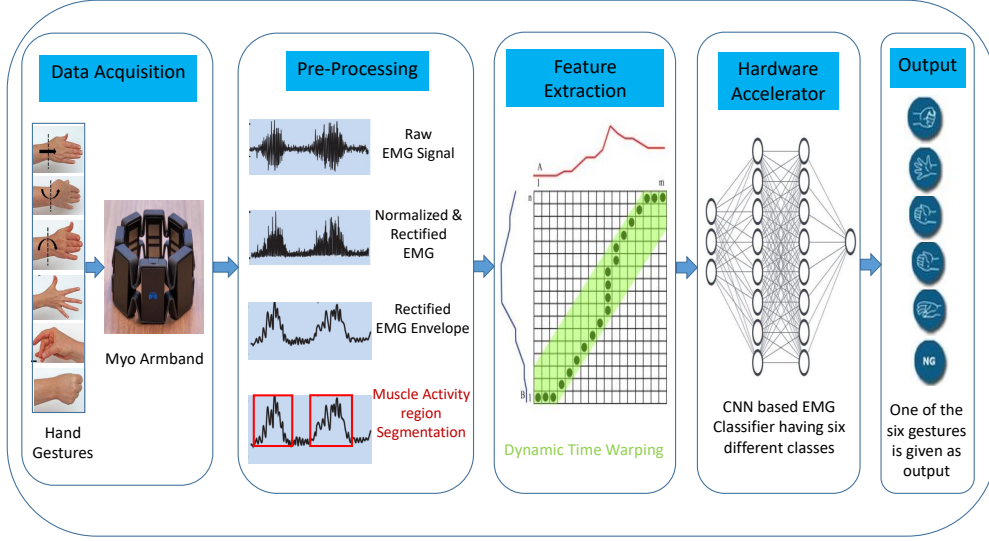


Figure 1-1: Hand Gesture Recognition System

because the inference phase does much less computing. The training phase must be done only once, while the inference phase runs whenever and wherever we need to use the network to make predictions or inferences. We have obtained the parameters using software to develop a CNN for the inference phase.

1.1 1D Convolution

1.1.1 1D Convolution with 1D Input

To obtain local features from sequential data, we perform 1D convolution. A filter having 1D kernel is slid over the input array when applied to a one-dimensional input, like a time series signal like Emg signal. The output feature map contains a single value that is the result of the kernel doing an element-wise multiplication with the relevant segment of the input at each place, that depends upon stride. The output's size can be adjusted with optional padding, and the kernel maps to the input with a predefined stride. Signal processing and deep learning applications which extract features from temporal sequences, and frequently use this kind of convolution.

Example of Computation in 1D Convolution over 1D Input (Kernel = 2, Stride = 2)

Input Vector:

1 3 2 4 0 1 5 2 1 3

Kernel:

1 -1

Convolution Operation:

Convolution is performed by sliding the kernel over the input vector and computing the dot product at each position. Given: - Kernel size = 2 - Stride = 2

This means the kernel moves 2 positions to the right after each operation. At each step:

$$\text{Output} = (k_1 \cdot x_1) + (k_2 \cdot x_2)$$

where k_1, k_2 are kernel values and x_1, x_2 are input values under the kernel window.

Output Vector:

-2 -2 -1 3 -2

Kernel Input-Output Table:

Step	Input (x_1, x_2)	Kernel (k_1, k_2)	Output
1	(1, 3)	(1, -1)	$(1 \times 1) + (3 \times -1) = -2$
2	(2, 4)	(1, -1)	$(2 \times 1) + (4 \times -1) = -2$
3	(0, 1)	(1, -1)	$(0 \times 1) + (1 \times -1) = -1$
4	(5, 2)	(1, -1)	$(5 \times 1) + (2 \times -1) = 3$
5	(1, 3)	(1, -1)	$(1 \times 1) + (3 \times -1) = -2$

1.1.2 1D Convolution with 2D Input

When we have multi-channel data to extract features, the input can be represented as a two-dimensional array with dimensions (W, C) , where C is the number of channels and W is the time stamps of the sequence. In this case, 1D convolution can use one-dimensional or multi-dimensional kernel, but the kernel slides over the input only in one direction, mainly it slides across time window. At each step, the kernel computes a dot product across all the channels and spatial dimensions, producing an output value that captures patterns along different time stamps of the input but also across channels. This is particularly useful in deep learning models where input data is coming from multi-channel sensor signals, word embeddings, or biomedical signals require learning from both spatial and cross-channel correlations. The resulting feature maps extracts local dependencies across multiple input dimensions, enabling more better feature extractions.

Example of computation in 1D convolution on 1D input (Time Steps $\times$ Channels)

Input Matrix (8×6)

1	0	2	1	3	1
2	1	0	2	1	0
1	2	1	1	0	3
0	1	2	0	2	1
3	0	1	2	1	2
2	1	1	1	0	1
1	2	0	0	3	2
0	1	1	2	1	0

Kernel (2×6)

1	-1	0	1	0	-1
0	1	-1	0	1	1

Convolution Equation

$$y[t] = \sum_{i=0}^1 \sum_{j=0}^5 \text{Input}[t+i][j] \cdot \text{Kernel}[i][j]$$

Stride = 2, Output Length = 4

Step-by-step Computation

Step 1:

$$\begin{aligned} y[0] &= 1 \cdot 1 + 0 \cdot (-1) + 2 \cdot 0 + 1 \cdot 1 + 3 \cdot 0 + 1 \cdot (-1) \\ &\quad + 2 \cdot 0 + 1 \cdot 1 + 0 \cdot (-1) + 2 \cdot 0 + 1 \cdot 1 + 0 \cdot 1 \\ &= 1 + 0 + 0 + 1 + 0 - 1 + 0 + 1 + 0 + 0 + 1 + 0 = \boxed{3} \end{aligned}$$

Step 2:

$$\begin{aligned} y[1] &= 1 \cdot 1 + 2 \cdot (-1) + 1 \cdot 0 + 1 \cdot 1 + 0 \cdot 0 + 3 \cdot (-1) \\ &\quad + 0 \cdot 0 + 1 \cdot 1 + 2 \cdot (-1) + 0 \cdot 0 + 2 \cdot 1 + 1 \cdot 1 \\ &= 1 - 2 + 0 + 1 + 0 - 3 + 0 + 1 - 2 + 0 + 2 + 1 = \boxed{-1} \end{aligned}$$

Step 3:

$$\begin{aligned} y[2] &= 3 \cdot 1 + 0 \cdot (-1) + 1 \cdot 0 + 2 \cdot 1 + 1 \cdot 0 + 2 \cdot (-1) \\ &\quad + 2 \cdot 0 + 1 \cdot 1 + 1 \cdot (-1) + 1 \cdot 0 + 0 \cdot 1 + 1 \cdot 1 \\ &= 3 + 0 + 0 + 2 + 0 - 2 + 0 + 1 - 1 + 0 + 0 + 1 = \boxed{4} \end{aligned}$$

Step 4:

$$\begin{aligned} y[3] &= 1 \cdot 1 + 2 \cdot (-1) + 0 \cdot 0 + 0 \cdot 1 + 3 \cdot 0 + 2 \cdot (-1) \\ &\quad + 0 \cdot 0 + 1 \cdot 1 + 1 \cdot (-1) + 2 \cdot 0 + 1 \cdot 1 + 0 \cdot 1 \\ &= 1 - 2 + 0 + 0 + 0 - 2 + 0 + 1 - 1 + 0 + 1 + 0 = \boxed{-2} \end{aligned}$$

Final Output Vector

3	-1	4	-2
---	----	---	----

1.2 Max Pooling

1.2.1 1D Max Pooling

1D Max Pooling is mainly used for downsampling operation to reduce the temporal dimension of data(feature maps and) In this operation, a pooling window of fixed size slides over the input sequence, and at each step, the maximum value within the window is selected and is given as output. This operation reduces further computations in the neural network. When applied to a 1D input, max pooling involves scanning the input vector with a defined window size and stride, selecting the maximum value in each window.

1.2.2 2D Max Pooling

For a 2D input with shape (C, W) , max pooling is applied independently to each channel across its width, which helps in preserving the channel-wise structure of the input while reducing dimension. Max pooling is especially effective after convolutional layers, as it emphasizes the most important features and helps the model focus on the most relevant features.

Example of computation in Maxpooling

1D Max Pooling

Input vector: $[1, 3, 2, 4, 5, 0]$

Kernel size: 2, Stride: 2

Window Index	Window Elements	Maximum Value
0	$[1, 3]$	3
1	$[2, 4]$	4
2	$[5, 0]$	5

Output vector: $[3, 4, 5]$

2D Max Pooling

Input matrix (shape 2×6):

$$\begin{bmatrix} 1 & 3 & 2 & 4 & 5 & 0 \\ 2 & 1 & 3 & 0 & 4 & 6 \end{bmatrix}$$

Kernel size: 2, Stride: 2

Channel 1:

Window Index	Window Elements	Maximum Value
0	[1, 3]	3
1	[2, 4]	4
2	[5, 0]	5

Channel 2:

Window Index	Window Elements	Maximum Value
0	[2, 1]	2
1	[3, 0]	3
2	[4, 6]	6

Output matrix:

$$\begin{bmatrix} 3 & 4 & 5 \\ 2 & 3 & 6 \end{bmatrix}$$

1.3 Dense Layer

A dense layer is an important component in neural networks where each input neuron is connected to every output neuron through a set of learnable weights, because its used at the output to converge the features extracted in previous layers. In the context of 1D data, such as sequences or flattened features from convolutional layers, the dense layer takes a one-dimensional vector as input and transforms it into another vector of specified length, by matrix multiplication with weight matrix. This transformation is achieved by computing a weighted sum of the inputs followed by the addition of a bias term, and typically applying a sigmoid or softmax functions to generate output. We can show this mathematically as follows, for an input vector $\mathbf{x} \in \mathbb{R}^n$ and weight matrix $W \in \mathbb{R}^{m \times n}$, the output is given by $\mathbf{y} = f(W\mathbf{x} + \mathbf{b})$, where f is the activation function and $\mathbf{b}$ is the bias vector.

1.3.1 Importance of Dense Layers

Dense layers are necessary for predicting or converging complex and non-linear combinations of features extracted by preceding layers such as convolutional or pooling layers. In 1D neural architectures, especially those used in time series analysis, natural language processing, and audio recognition, dense layers act as the decision-making units by integrating and interpreting the high-level features learned earlier in the network during convolution. They are commonly placed near the output of the model to map learned representations to the desired output size, such as scores in classification tasks. The fully connected nature of dense layer allows them to capture dependencies in the data, making them an important component for output prediction.

Example of computation in Dense Layer

Given

- Input vector:

$$\begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

- Weight matrix:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

- Bias vector:

$$\begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Step 1: Matrix Multiplication

Multiply the weight matrix with the input vector:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \cdot \begin{bmatrix} 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 1 \cdot 2 + 2 \cdot 3 \\ 3 \cdot 2 + 4 \cdot 3 \end{bmatrix} = \begin{bmatrix} 2 + 6 \\ 6 + 12 \end{bmatrix} = \begin{bmatrix} 8 \\ 18 \end{bmatrix}$$

Step 2: Add Bias

Now add the bias vector:

$$\begin{bmatrix} 8 \\ 18 \end{bmatrix} + \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 9 \\ 17 \end{bmatrix}$$

Final Output

$$\text{Output vector} = \begin{bmatrix} 9 \\ 17 \end{bmatrix}$$

1.4 Activation Functions

Activation functions are mathematical functions applied to the output of convolution in neural networks because they introduce non-linearity into the model so that it can learn complex patterns and relationships within the data. Without activation functions, a neural network would simply behave like a linear model regardless of its depth. Activation functions play a crucial role in controlling the flow of useful information and gradients through the neural network.

1.4.1 ReLU Activation Function

The Rectified Linear Unit (ReLU) is one of the most widely used activation functions in modern deep learning architectures. It is defined as:

$$f(x) = \max(0, x)$$

This function outputs the input directly if it is positive; otherwise, it outputs zero. ReLU is computationally efficient and helps mitigate the vanishing gradient problem often seen in deep networks. By zeroing out negative values, ReLU introduces sparsity, which can improve model generalization.

1.4.2 Tanh Activation Function

The hyperbolic tangent (Tanh) activation function maps input values to the range $(-1, 1)$ and is defined as:

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Tanh is a smooth, continuous, and differentiable function in comparison to ReLu . The main idea behind using Tanh model both positive and negative activations, which often helps in capturing symmetrical relationships in informational data. However, Tanh can still suffer from the vanishing gradient problem for large absolute input values, where the gradient becomes very small, slowing down training.

Chapter 2

Motivation

The main motivation of this work is to develop a real-time EMG-based gesture recognition system for edge devices. Gesture recognition system plays an important role in human-machine interaction, forming backbone of many next-generation technologies such as smart wearables, augmented and virtual reality (AR/VR), prosthetics, robotics, As wearable devices become are mobile and user-centric, we need something and that can perform gesture recognition at the edge hence, there is a growing demand for interfaces that have low-latency, and power efficient. Hand gestures, being a most common form of non verbal for of human communication, are ideally suited for this role.

In the era of edge-based hand gesture recognition, we need to select a suitable hardware for computing so that it can full fill the stringent requirements of real-time responsiveness and energy efficiency. Traditional CPUs, designed for versatility and general-purpose computing, often fall short in these contexts because they have inherently higher latency and limited parallel processing capabilities. This results in slower inference times and inconsistent performance in case of edge devices, which are unacceptable for applications requiring immediate feedback based upon gestures identified, such as wearable devices .

On the other hand, GPUs are architected for highly parallel, greater throughput so that can deliver significantly faster speeds compared to CPUs. However, their advantages come with trade-offs that make them less suitable for edge devices, i.e. high power consumption ,which we cannot afford in case of battery-powered wearables. In addition to that, GPUs tend to have larger physical footprints, which makes it difficult to integrate

them into compact, lightweight wearable devices.

An efficient substitute that offers the best possible combination between power efficiency and computing performance for edge deployments is Field-Programmable Gate Arrays (FPGAs). In contrast to general-purpose processors or fixed-function accelerators, FPGAs enable hardware customization at the logic level, allowing for exact optimization for the particular machine learning models employed in gesture recognition. This includes task-specific parallelism that takes advantage of the natural structure of neural network operations and bit-level precision tuning that can increase throughput and lower power consumption without compromising accuracy.

Moreover, FPGAs inherently provide low latency processing due to their pipelining and parallelism, which ensures fast response for times—critical for applications where lag can disrupt user experience. With typically low power consumption, FPGAs are well-suited for continuous operation for longer hours in wearable devices. Their small footprint and ability to integrate processing, memory, and I/O functions on a single chip further simplify system design, reduce overall size, and lower costs.

To summarize, FPGAs are the perfect hardware platform for real-time, low-power hand gesture detection systems at the edge because of their distinctive blend of adaptable design, energy economy, consistent execution, and small form size. By utilizing these benefits, wearable technology may be developed that is accurate, responsive, and useful for daily usage, greatly expanding the field of human–machine interaction.

Aspect	Hardware Accelerator (FPGA)	CPU	GPU
Latency	Very low	Moderate to high	Moderate
Power Consumption	Very low	High	Very high
Throughput	High	Low to moderate	High
Customizability	Fully customizable	Not customizable	Limited
Size / Integration	Small footprint	Larger	Bulky

Table 2.1: Comparison of FPGA, CPU, and GPU for Hand Gesture Recognition at the Edge

Chapter 3

Background Study and Related works

3.1 Reinforcement Learning-Based Hand Gesture Recognition Systems Using EMG and IMU Signals [1]

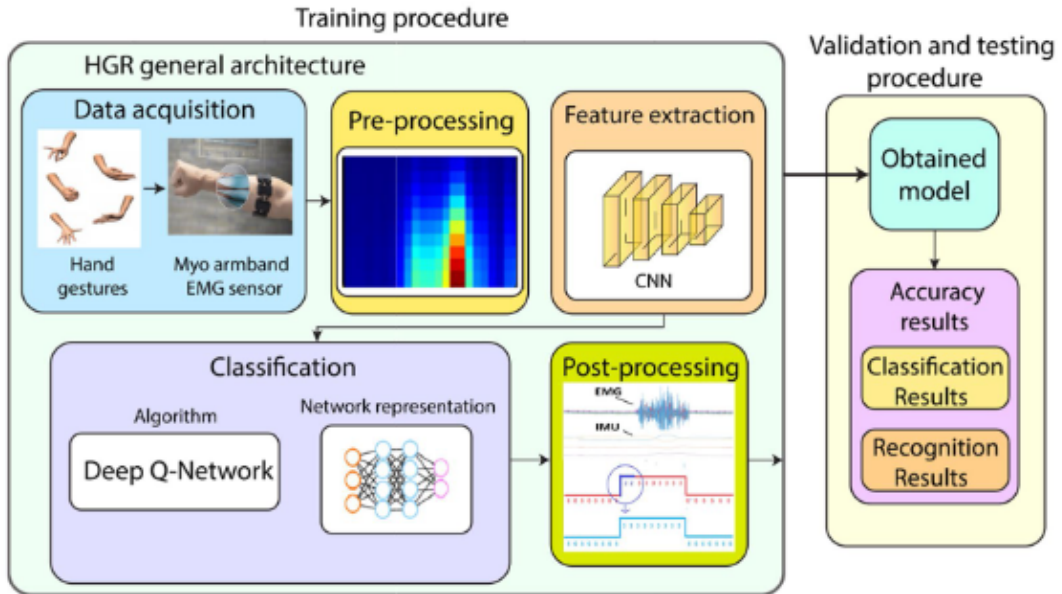


Figure 3-1: HGR system utilizing EMG-IMU signals and deep neural network architecture for gesture classification and robotic control [1]

In order to manage robotic platforms, this work introduces a reinforcement learning-based hand gesture recognition (HGR) system as shown in figure.3-1 that uses EMG and IMU inputs from a Myo Armband. Six hand motions are classified by the system using a Deep Q-Network (DQN) with a low inference time (20 ms) and high accuracy (97.45percent).

The HGR system exhibits quick, precise, and efficient control by effectively operating a 3-DOF tandem helicopter and a virtual 6-DOF UR5 robot. By utilizing sensor data and reinforcement learning techniques to enable adaptive, real-time gesture-based control of robotic systems, the method improves human-machine interactions.

The physical modeling and control methods for the robots are important parts of the control architecture as a whole. Two rotors produce forces that move the helicopter. These forces are added together to get total thrust and differential force, which are used to model the helicopter’s dynamics. Taking into account the geometry and inertia of the system, equations connect these forces to the helicopter’s angular speeds along the travel, elevation, and pitch directions. For each degree of freedom, a PID control scheme is used, with cascade control loops for travel, elevation, and pitch. To improve speed, features like reset and anti-windup are added. It is possible to use simple operation modes because control inputs are made based on gesture commands like wave out, wave in, open, and pinch, or from IMU data. To make the virtual UR5 robot move in the way that is wanted, a PID-based tracking controller is used along with a kinematic model based on Denavit-Hartenberg parameters. This is because dynamic modeling is too complicated to do on its own. The system as a whole combines physical models, sensor-based control, and virtual simulation to make controlling robotic and helicopter systems easy and accurate. The effectiveness of the HGR system is demonstrated through high classification accuracy (97 percent) and low mean squared errors in control tasks, indicating robust tracking, minimal overshoot, and quick settling times. The system’s low complexity and adaptability make it suitable for controlling various robotic platforms through intuitive hand gestures. Future work aims to improve gesture recognition accuracy and refine control strategies to enhance system robustness and responsiveness.

The comprehensive system for hand gesture recognition involves preprocessing EMG-IMU data through segmentation, filtering, and spectrogram extraction, followed by feature extraction using residual CNN blocks inspired by Inception modules. The DQN agent learns to recognize gestures by interacting with the environment, receiving rewards for correct predictions, and optimizing its policy via experience replay and target networks. Validation on user-specific datasets confirms high accuracy and robustness across different users. The recognized gestures generate control commands for UAV prototypes and robotic manipulators, demonstrating effective human-robot interaction.

In the control architecture for the virtual UR5 manipulator, forward kinematics, Jacobian matrices, and a minimum norm PID controller are employed. MATLAB models the kinematics, while CoppeliaSim simulates the environment, with communication facilitated via an API. The control approach involves calculating position and orientation errors, using the inverse Jacobian to determine joint velocities, and representing orientations through rotation matrices and Euler angles. Gesture commands derived from the HGR system are integrated into this framework, enabling intuitive and precise control of the robot. Validation results show high accuracy (97 percent) and robustness, with gesture-based commands facilitating movement and orientation adjustments effectively.

Overall, this body of research highlights the significant progress in integrating sensor-based gesture recognition, reinforcement learning, and advanced control strategies for robotic and UAV applications. The combination of deep learning techniques with physical modeling and simulation demonstrates the potential for developing intuitive, adaptive, and high-performance human-robot interfaces. Future directions include enhancing recognition accuracy, reducing latency, and expanding the range of gestures and control capabilities to facilitate broader real-world deployment. [1].

3.2 Neurobehavioral Development of Emotion-Specific Spontaneous Facial Responses in Infants [2]

This research provides a comprehensive examination of the developmental progression of spontaneous facial responses (SFRs) to emotional stimuli in infants, emphasizing both neural and behavioral dimensions. The study investigates how infants, during their first year of life, begin to perceive, interpret, and physiologically respond to various emotional facial expressions, specifically happiness, anger, and fear. By employing advanced neurophysiological recording techniques such as electromyography (EMG) and electroencephalography (EEG), the research aims to elucidate the underlying neural mechanisms that facilitate emotion recognition and social communication in early development.

The findings reveal a marked developmental shift between 4 and 7 months of age. At 4 months, infants demonstrate limited or non-specific facial muscle activity in response to emotional stimuli, indicating that their capacity for emotion-specific facial mimicry and

recognition is still emerging. These early responses are characterized by generalized facial activity that does not differentiate between distinct emotional expressions, reflecting the nascent stage of social-emotional processing. In contrast, by 7 months, infants exhibit clear, emotion-congruent facial responses. Specifically, increased activation of the zygomaticus major muscle (associated with smiling) is observed when infants view happy faces, while heightened activity of the frontalis muscle (associated with forehead raising) occurs in response to fearful expressions. These responses are particularly prominent within the 1000–3000 ms window following stimulus presentation, suggesting the involvement of more complex neurocognitive processes beyond simple motor mimicry.

The study underscores the importance of precise stimulus control and timing in capturing these responses, as well as the significance of neural maturation and experiential learning in shaping emotional responsiveness. The development of emotion-specific facial responses appears to be influenced by the maturation of neural circuits involved in emotion processing, such as the mirror neuron system, amygdala, and prefrontal regions. Additionally, social interactions and caregiver experiences play a crucial role in reinforcing and shaping these responses, facilitating the transition from perceptual recognition to more sophisticated emotional understanding and expression.

Furthermore, the research highlights that the emergence of emotion-specific facial responses is not solely a reflexive process but involves evaluative and interpretative mechanisms that integrate perceptual input with social and cognitive context. This complex interplay suggests that early emotional development is a dynamic process driven by neural maturation, experiential factors, and social environment. The findings contribute valuable insights into the neural and behavioral foundations of social cognition, emphasizing that the capacity for emotion recognition and expression develops gradually and is essential for effective social communication.

In conclusion, this study advances our understanding of early emotional development by demonstrating how infants' spontaneous facial responses evolve during the first year of life. It emphasizes the importance of neural maturation, experiential learning, and social interaction in fostering emotion-specific responses, which are fundamental for subsequent social and emotional competence. These insights have significant implications for identifying atypical developmental trajectories and informing interventions aimed at supporting social-emotional growth in infants at risk for developmental disorders. [2]

3.3 EMG Analysis of Tibialis Anterior and Gastrocnemius in ASD Gait [3]

This study conducted a detailed analysis of electromyographic (EMG) signals from the lower limb muscles, specifically the Tibialis Anterior (TA) and Gastrocnemius (GAS), during walking in children with Autism Spectrum Disorder (ASD) compared to typically developing (TD) children aged 6 to 13 years. Using wireless EMG sensors following standardized protocols, data were collected from 8 children with ASD and 10 TD children, and the EMG signals were normalized over the gait cycle to facilitate accurate comparison across subjects and phases of walking.

The key findings of the study highlight notable differences in muscle activation patterns between the two groups. During the midstance phase of gait, TD children exhibited a wider range of TA muscle activity, with higher EMG amplitudes indicating more effective dorsiflexion control during this phase. In contrast, children with ASD showed higher ankle plantar flexion activity, with mean EMG values around -7 microVolt in ASD children compared to approximately 29 microVolt in TD children, suggesting differences in push-off mechanics and ankle stability during walking.

Furthermore, during the midswing phase, children with ASD demonstrated greater ankle dorsiflexion, with mean EMG amplitudes around -19 microV, compared to -24 microV in TD children. This indicates altered neuromuscular control during the swing phase, potentially affecting foot clearance and gait efficiency. The study also found a significant difference in GAS muscle activity during walking, with a p-value of 0.042, confirming that TD children generally have higher plantar flexion activity during midstance, which is crucial for propulsion and stability.

Analysis of the EMG patterns revealed that children with ASD exhibit altered muscle activation across different gait phases, particularly in the GAS and TA muscles. These differences suggest that children with ASD may have atypical gait mechanics, possibly due to neuromuscular coordination deficits. The findings support the potential of EMG signals, especially from the GAS muscle, to serve as effective biomarkers for distinguishing gait abnormalities in ASD.

The study concludes by emphasizing the importance of these EMG differences in understanding gait impairments in children with ASD. It also recommends exploring upper

limb muscles in future research to gain a more comprehensive understanding of neuro-muscular control and coordination deficits, which could inform targeted rehabilitation strategies aimed at improving gait patterns and overall motor function in this population. Overall, the research underscores the value of EMG analysis in clinical assessments and the development of personalized intervention approaches for children with ASD [3].

3.4 FPGA-Based Acceleration of CNNs for Real-Time Digit Recognition [4]

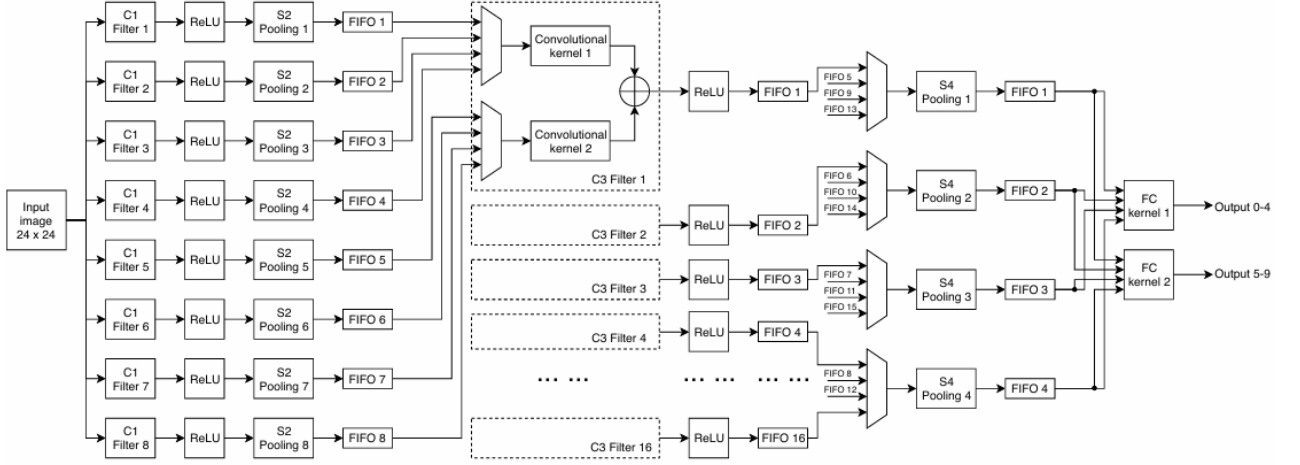


Figure 3-2: CNN Model used in digit recognition

This overview highlights the implementation of convolutional neural networks (CNNs) as shown in fig.3-2 in hardware, emphasizing the significant role of FPGA technology. FPGAs offer high degrees of parallelism, reconfigurability, and energy efficiency, making them ideal for embedded AI applications such as digit recognition. Their ability to accelerate inference processes while maintaining low power consumption surpasses traditional CPU and GPU solutions.

Designing FPGA-based CNN architectures involves optimizing resource utilization through pipelined processing, fixed-point arithmetic, and specialized dataflow strategies like weight stationary or row stationary approaches. These techniques enable high throughput, often reaching hundreds of thousands of images per second, suitable for real-time applications. A key advantage of FPGA deployment is the preservation of high inference accuracy despite

hardware constraints. Using fixed-point arithmetic with word lengths of around 13 bits or more ensures minimal accuracy loss, which is crucial for applications requiring precise results, such as medical diagnostics or security systems. Energy efficiency is another prominent benefit, with FPGA accelerators demonstrating power consumption levels significantly lower than GPUs and CPUs. This makes FPGAs particularly suitable for portable and power-sensitive environments. Additionally, their hardware flexibility allows for customization of neural network layers and operations, leading to further reductions in power and hardware footprint. FPGAs also excel in handling complex operations like large convolutional kernels, which are challenging for GPUs due to area limitations. This capability enables the deployment of more advanced CNN architectures that demand extensive computation. The reconfigurability of FPGA hardware supports ongoing updates and optimizations, ensuring adaptability to evolving AI models and applications. In conclusion, FPGA technology provides a compelling platform for high-accuracy, energy-efficient, and scalable CNN inference. Its combination of high performance and flexibility makes it a cornerstone for embedded AI systems requiring real-time processing within resource-constrained environments. Ongoing advancements continue to expand FPGA capabilities, promising even greater efficiency and performance in future AI deployments [4].

3.5 Portable Myoelectric Control Systems Using Quantized CNNs on FPGA Hardware [5]

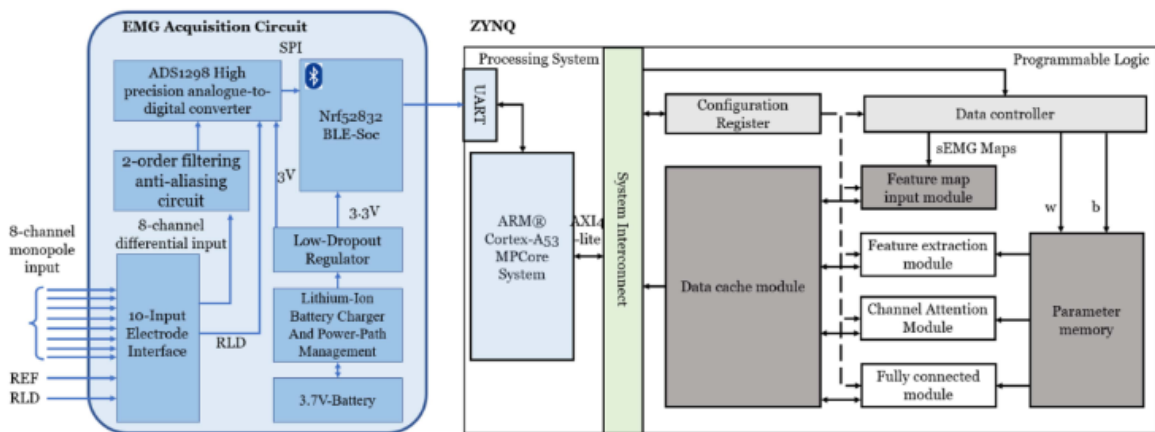


Figure 3-3: Hardware architecture used in this work [5]

Recent research in surface electromyography (sEMG) gesture and force recognition systems has focused on developing low-power, real-time, and portable solutions, primarily utilizing FPGA-based hardware accelerators. These systems leverage lightweight deep learning models, such as optimized convolutional neural networks (CNNs), which incorporate advanced modules like ultra-lightweight depthwise separable convolutions (UL-DSC), channel attention mechanisms with global average pooling (CA-GAP), and quantization techniques to minimize model size and computational complexity.

The typical workflow involves preprocessing sEMG signals through filtering, segmentation, and transformation into spectrograms via Short-Time Fourier Transform (STFT). These spectrograms are then fed into dedicated hardware accelerators designed for efficient classification. For example, some architectures achieve inference times as low as 41.9 microseconds per frame, with recognition accuracy reaching approximately 94.92%, and model sizes as small as 0.026 MB containing around 5,000 parameters. Such models are optimized for deployment on embedded FPGA platforms like Zynq, enabling real-time operation with minimal power consumption (0.317 W), making them suitable for wearable devices, prosthetic control, and robotic applications.

The hardware architectures shown in figure 3-3 integrate sEMG acquisition circuits with filtering modules, Bluetooth communication interfaces, and specialized recognition accelerators. These accelerators utilize modular processing elements (PEs) optimized for convolutional operations, including depthwise and pointwise convolutions, and are configurable to balance accuracy and resource utilization . Quantization strategies, primarily 8-bit integer quantization, are employed post-training to further reduce memory footprint and power consumption without significantly sacrificing accuracy .

Research efforts also emphasize improving model robustness and generalization across different users and conditions. Expanding datasets, enhancing cross-subject robustness, and exploring traditional machine learning methods are ongoing to improve system reliability. Additionally, activation functions like Leaky-ReLU are used to enhance non-linear modeling capabilities and prevent issues such as vanishing gradients, thereby improving overall performance.

Overall, these advancements demonstrate that FPGA-based systems with lightweight, optimized neural networks can deliver high-accuracy, low-latency, and energy-efficient solutions for EMG-based gesture and force recognition. The integration of hardware-efficient models,

signal preprocessing, and dedicated accelerators paves the way for practical, portable myoelectric control systems applicable in biomedical, prosthetic, and human-computer interaction domains . Future directions include expanding datasets, improving robustness across diverse populations, and exploring hybrid approaches combining traditional machine learning with deep learning techniques to further optimize performance and resource utilization [5].

Chapter 4

EMG Classifier design

4.1 Proposed Approach 1

This thesis presents two hardware models, which are machine learning-based hardware accelerators for real-time hand gesture recognition using EMG signals. The designed hardware supports cross-user operation and processes sensor input in real time during the inference phase. Model 1 approach is described below:

4.1.1 Model 1: (Single Channel)

Input: 1-channel 1D EMG signal (after preprocessing and performing DTW on the continuous time-series Emg)

1. **CNN Training:** The CNN is trained offline on a labeled EMG dataset of 71 users to learn the patterns.
2. **Conv1D Layer 1 + ReLU:** In the Conv1D layer, the input EMG signal is passed through 32 filters with a kernel size of 2 to perform 1D convolution on 1D time series data to extract features. The convolution uses weights stored in Block RAM (BRAM). The output is activated by a ReLU function.
3. **MaxPooling1D:** Max pooling with pool size 2 downsamples the feature maps obtained from the conv1D layer 1 to reduce temporal dimensions.
4. **Buffering Intermediate Output:** The pooled outputs are stored in shift registers so that it can be used next by the convolutional layer.

5. **Conv1D Layer 2 + ReLU:** Using a single convolutional unit reused across 64 filters, the buffered data is convolved with weights selected by a multiplexer from 64 BRAMs. The output is activated by ReLU to introduce nonlinearity.
6. **Feature Aggregation:** The convolution outputs are aggregated using an adder block to form the input for the dense layer 1, having weight matrix of 64 x 64.
7. **Dense Layer 1:** The aggregated features are passed through 64 neurons where Serial-In Parallel-Out (SIPO) converters facilitate efficient weight loading and parallel matrix-vector multiplication.
8. **Dense Layer 2:** The output of Dense Layer 1 is passed through 6 neurons to generate 6 logits.
9. **Hardmax Activation:** The logits are directly converted to a one-hot vector output by selecting the class with the highest score using a comparator, effectively bypassing the probabilistic interpretation of softmax, as softmax uses extensive hardware.
10. **Output Prediction:** The class corresponding to the highest logit value is selected as the predicted gesture output. No weight updates occur during inference.

4.1.2 CNN Architecture Model 1

Building the CNN model can be broadly divided into the following steps.

- Data Pre-Processing
- Feature extraction
- Model Training
- Testing the trained model

Pre-Processing

During data pre-processing, the entire EMG dataset is normalized and rectified. The envelope for each channel of the rectified signal is extracted using a 5th-order low-pass Butterworth filter. Spectrogram of the sum of these envelopes is then utilized to identify the region corresponding to muscle activity.

Feature extraction

Features are extracted for the identified muscle activity regions using Dynamic Time Warping (DTW).

Dynamic Time Warping (DTW) DTW is a method to estimate the similarity between time series by aligning them while accounting for both global and local shifts in the time dimension, providing a distance measure for time series classification.

The EMG time series that is closest to all other samples within a particular gesture class is selected using DTW and serves as the representative for that gesture. As a result, six representative time series, one for each gesture, are obtained and subsequently used as references to calculate feature vectors for each time series in the dataset (using DTW).

Model Training

In this analysis, data from 71 users is used to train the model. For each user, feature vectors (computed by calculating the DTW distance between each class representative and each pre-processed EMG signal) along with the one-hot encoded labels, are concatenated across all 71 users and used to train the below described neural network.

The model is also implemented using fixed-point parameter representation (Integer Weights and biases) of the default floating-point parameters to reduce computational complexity in the hardware realization of the model.

In the designed CNN as shown in figure 4-1, the input is given to a 1D convolutional layer comprising 32 filters with a kernel size of 2 and relu activation, followed by a 1-D max-pooling layer with a pool size of 2 that subsamples and reduces the dimensionality. Next, a second 1-D convolutional layer with 64 filters and a kernel size of 2 with relu activation is applied. The output from this layer is flattened and then passed to a Dense layer with 64 neurons and relu as activation layer, followed by a final Dense layer with 6 neurons and softmax activation for

Testing the trained model

The trained model is evaluated on data from 21 users, achieving an accuracy of 81.49% with the default floating-point parameters and 81.36% when using fixed-point parameter

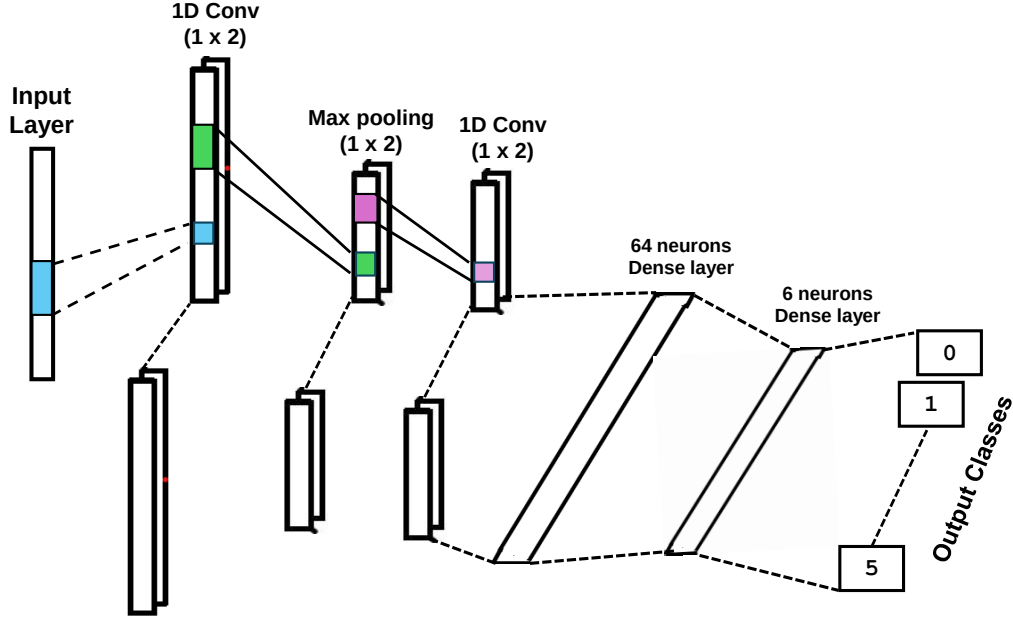


Figure 4-1: Neural Network Architecture used in this approach

representations.

While models such as LSTM, GRU, and Transformer excel at handling long sequences, they present significant challenges for edge devices. Transformers, in particular, require a large number of parameters (especially within their multi-head attention layers) and demand substantial computational resources, high memory usage, and typically rely on GPUs. In contrast, the sequential processing nature of LSTMs and GRUs limits parallelism, which can increase inference latency on less powerful hardware. Consequently, these architectures are generally less suitable for edge applications where processing power, memory, and energy resources are limited. Therefore, we implemented our model using 1D Convolution and Dense layers, which are more efficient and better suited for deployment on edge devices.

Numerous models have been developed to predict movements using EMG signals, still building a fully functional and reliable model for real-time applications remains a major challenge.

User-specific models, which are tailored to an individual's data, often achieve high accuracy because of their personalized nature. In contrast, creating general recognition models using EMG signals poses greater difficulties. General models require training only once

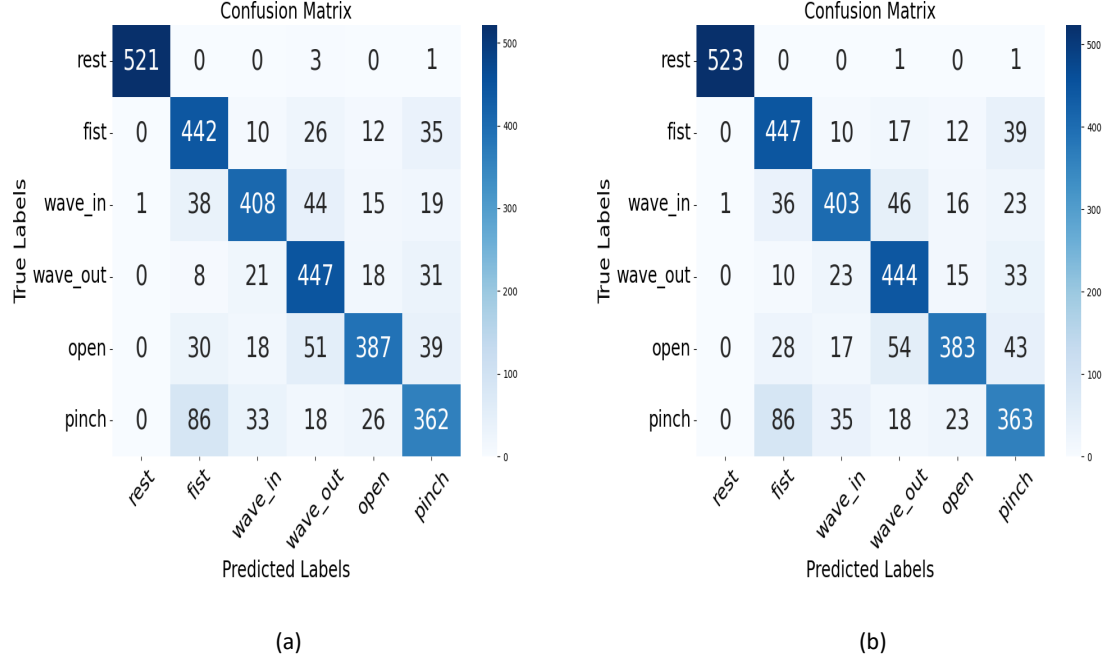


Figure 4-2: Confusion Matrices: Test Accuracy of the CNN Model 1(single channel) using (a) Floating-point parameters and (b) Fixed-point (Integer) parameters.

and can be applied to any user, whereas user-specific models must be trained for each individual and need retraining for different users.

A highly accurate and generalizable Human Gesture Recognition (HGR) model is essential for seamless interactions with electronic devices in applications such as prosthetic hand control, robotics, human-computer interaction (e.g., mouse control), gaming, sign language interpretation, virtual reality, and various medical tasks.

4.1.3 Dataset

The EMG-EPN-612 dataset is used in this analysis. It contains electromyography (EMG) signals from 612 individuals and is designed for benchmarking hand gesture recognition systems. EMG signals were recorded from the forearm as participants performed five distinct hand gestures— wave-in, wave-out, pinch, open, fist along with recordings taken when the hand was in a relaxed state.

We trained our model using data from 71 users, with each user contributing 150 samples (25 samples per gesture). This amounts to a training dataset of 150×71 samples. The test dataset consists of data from 21 users or 21×150 samples.

4.2 Limitations of Single-Channel Input and Motivation for Multi-Channel Architecture

Model 1, based on a single-channel EMG input, demonstrated limitations when tested on data from previously unseen users. Specifically, it achieved an accuracy of only **81.36%**, which was considerably lower than expected for reliable real-world performance. This result highlighted the model’s limited generalization capability, largely due to the restricted spatial information provided by a single EMG channel.

To address these shortcomings, we transitioned to Model 2, which utilizes a multi-channel input architecture. By incorporating data from all six EMG channels, Model 2 provides a richer and more comprehensive representation of muscle activity. This change enables better feature extraction, improved robustness across users, and significantly enhanced classification performance.

4.3 Proposed Approach 2

4.3.1 Model 2:(Multi channel)

To improve the accuracy achieved by the earlier model, we transitioned to Model 2, which takes 2D inputs from multiple EMG channels. The previous model, using only a single-channel input, achieved a lower accuracy of **81.36%**, indicating limited generalization and insufficient feature representation. Model 2 addresses this limitation by leveraging data from six EMG channels, allowing the network to extract more comprehensive spatial and temporal features, ultimately enhancing recognition performance.

Input: Multi-channel EMG signal $X \in \mathbb{R}^{1019 \times 6}$ (6 channels, 1019 time steps)

1. **Layer 1 – 1D Convolution + ReLU:** A set of 32 filters, each of size 2×6 , is applied to extract local temporal patterns across all channels. The convolution output passes through a ReLU activation function, resulting in an output shape of 32×1018 .
2. **Max Pooling:** Max pooling with a window size of 2 is applied along the temporal dimension, reducing the output size to 32×509 by downsampling.

3. **Layer 2 – Deep Convolution + ReLU:** The output from the max pooling unit is passed through 64 filters of size 32×2 to extract higher-level temporal features. The output is activated by ReLU and has a shape of 64×508 .
4. **Global Average Pooling (GAP):** The feature maps are averaged across all 508 time steps, producing a compact 64×1 feature vector.
5. **Fully Connected Layers:** This feature vector is passed through two dense layers to converge the features: first, a layer with 64 neurons applying weights of size 64×64 , followed by a second layer with 6 neurons applying weights of size 6×64 . The final output is a vector of 6 logits representing class scores.
6. **Hardmax Activation:** Instead of softmax, a comparator module (hardmax) selects the class with the highest logit value directly as the predicted gesture class to reduce hardware usage.

Output: Predicted gesture class from 6-channel EMG data using the trained CNN hardware accelerator with hardmax output.

4.3.2 CNN Architecture Model 2

The development and analysis of the Convolutional Neural Network (CNN) model for EMG-based hand gesture recognition can be broadly divided into the following steps:

- Pre-Processing
- Muscle Activity Detection
- Principal Component Analysis (PCA)
- Model Training
- Testing the Trained Model

Pre-Processing

During data pre-processing, the complete EMG dataset undergoes normalization and rectification. The envelope of each rectified signal channel is extracted using a low-pass Butterworth filter. This step enhances signal clarity and reduces high-frequency noise components, enabling more accurate downstream processing.

Muscle Activity Detection

The envelopes obtained from all six channels are produce envelope. A spectrogram of this total envelope is computed to identify the indices corresponding to muscle activity within the input signal. The segment of the signal corresponding to these indices is extracted and retained for further analysis.

Principal Component Analysis

Principal Component Analysis (PCA) is applied to the EMG data extracted in the previous step, this mainly used in multichannel model, to extract meaningful features from multi-channel data that capture the most important patterns or trends. During the training phase, PCA is used to determine the optimal number of principal components that capture a target level of variance. This dimensionality reduction technique helps retain the most relevant features while minimizing computational complexity.

Model Training

The CNN model is trained to classify six different hand gestures using input data that has been quantized and reduced via PCA. Labels are one-hot encoded. The training dataset includes samples from 71 participants, each contributing 150 samples (25 per gesture), resulting in a total of 150×71 samples.

To enable efficient deployment on hardware, the trained model is also implemented using integer-based fixed-point representations for weights and biases, replacing default floating-point parameters. This significantly reduces computational demand and is well-suited for embedded systems.

Testing the Trained Model

The trained CNN model is evaluated on data collected from 21 users. It achieves a recognition accuracy of 95.80% using default floating-point parameters and 95.49% when using fixed-point parameter representations. These results demonstrate the model's robustness and hardware efficiency with minimal accuracy loss.

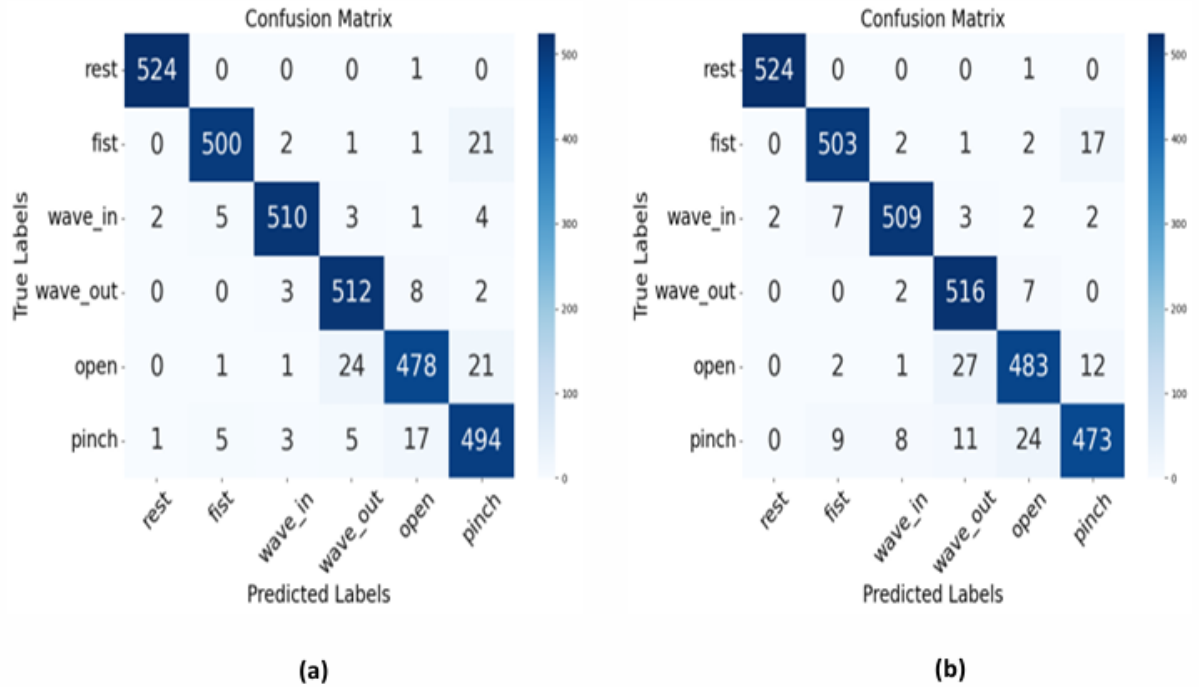


Figure 4-3: Confusion Matrices: Test Accuracy of the CNN Model 2(six channels) using (a) Floating-point parameters and (b) Fixed-point (Integer) parameters.

Chapter 5

Hardware Implementation

5.1 Implementation of Model 1

5.1.1 Architecture of Model 1 CNN

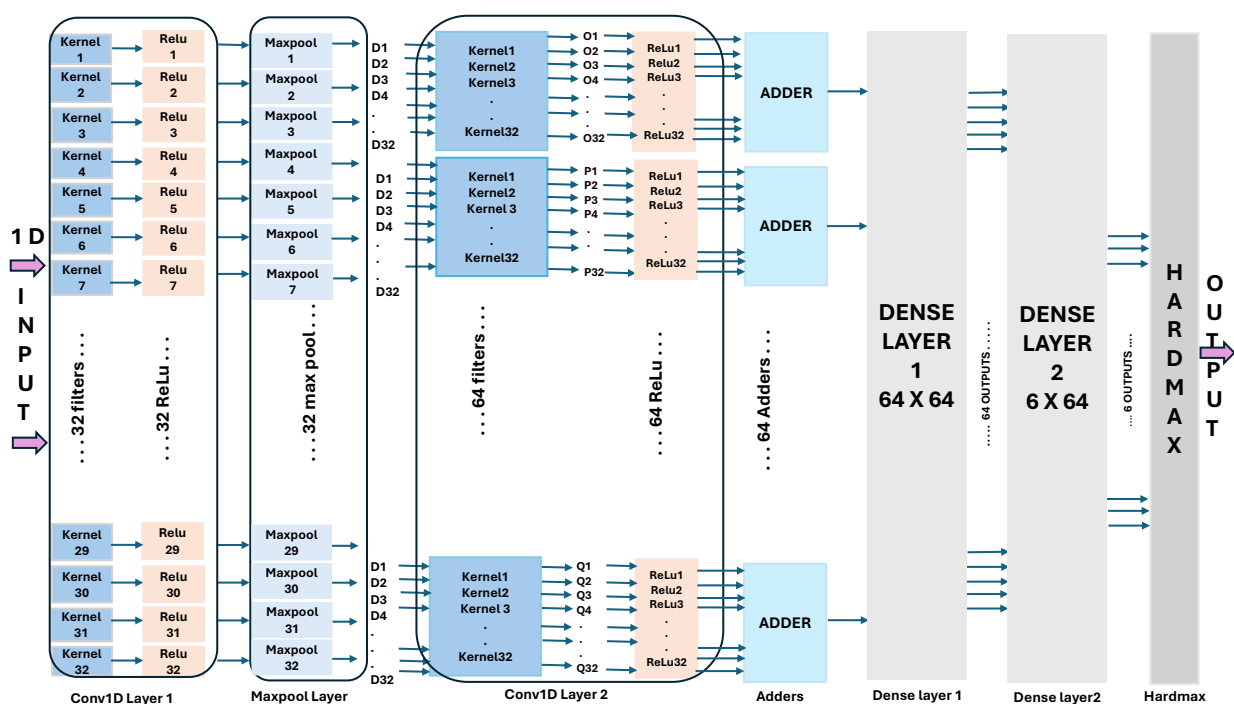


Figure 5-1: Block diagram of Architecture of Model 1

The overall architecture of the design includes model training using EMG signals and gesture recognition using a hardware classifier, which comprises a CNN, will be used for the testing phase. We have designed a CNN model 1 as depicted in figure 5-1 using Verilog HDL, to implement the inference phase using learned parameters, which will be used for

gesture recognition, the detailed specifications of each layer of CNN architecture proposed are given below:

- **Conv1D Layer1:** Filters = 32, Kernel size = 2, Activation = ReLU, parameters = 96.
- **MaxPooling1D:** Pool size = 2.
- **Conv1D Layer2:** Filters = 64, Kernel size = 2, Activation = ReLU, parameters = 4160.
- **Flatten Layer:** Basically, adders are used to aggregate and convert features into a 1D vector.
- **Dense Layer1 and Dense Layer2:** Dense layer1(Units = 64 ,parameters =4160) and (Units = 6 , parameters =390) respectively.
- **Output layer:**Hardmax.

The complete Convolutional Neural Network (CNN) architecture for Model 1 is shown in figure5-1 is designed to efficiently process Electromyography (EMG) signals from 1 channel for gesture recognition applications. The model follows a modular structure, where each layer is optimized for latency, memory usage, and hardware reusability. The architecture comprises of two convolutional layers, two dense layers, and a final HardMax-based classification unit. The use of shift registers, multiplexers, and Serial-In Parallel-Out (SIPO) structures ensures seamless data transfer across layers, using handshaking mechanisms that will enable real-time processing.

- **Input:** The input to the model consists of preprocessed EMG signals(1D) from one channel for six time stamps, upon which 1D convolution has to be performed.
- **Conv1D Layer 1:** This layer consists of **32 filters**, each of kernel size 2. Convolution operations are implemented using Multiply-Accumulate (MAC) units, where each kernel slides over input sequence depending upon stride and perform convolution. The outputs of the convolution are passed through a **ReLU activation function**, introducing non-linearity into the network. To reduce the spatial dimension and retain essential features, a **MaxPooling** operation is performed with a pool size of 2. This reduces the feature map size by half, while preserving the dominant activation values.

- **Data Storage for Reuse:** The MaxPooled output from Conv1D Layer 1 is stored in **dedicated shift registers**, allowing the model to reuse the data during the execution of convolution in Conv1D Layer 2. An address counter manages the indexing of these registers, which can be controlled to ensure that the correct data is accessed with the convolution cycle of Layer 2.
- **Conv1D Layer 2:** This layer uses **64 convolutional filters**, each requiring a separate set of weights. Due to hardware resource constraints, only **one convolution unit** is physically implemented and reused over **64 clock cycles**. A **multiplexer (MUX)** selects the appropriate weight vector (loaded from BRAM via shift registers) and routes it to the convolution unit at each clock cycle. This reuse strategy minimizes LUT and DSP block utilization on FPGA hardware while maintaining throughput. The outputs are again passed through a **ReLU activation**, followed by an **adder block** that performs local summation to facilitate feature aggregation.
- **Flatten and SIPO:** The output of Conv1D Layer 2, which is inherently sequential, is flattened into a one-dimensional vector. This flattened vector is fed into a **Serial-In Parallel-Out (SIPO)** register structure, converting the serial data stream into a parallel vector. This conversion is essential for supporting simultaneous computations in the dense layers that follow.
- **Dense Layer 1:** This layer performs a 64×64 **matrix-vector multiplication**. To achieve real-time performance, **64 parallel MAC units** are instantiated, allowing each row of the weight matrix to be processed simultaneously. Weights are preloaded from BRAM blocks into local registers for fast access. The output of this layer undergoes **ReLU activation**, enabling the model to learn non-linear decision boundaries. Another SIPO structure is used to parallelize the output vector for the final dense layer.
- **Dense Layer 2:** The second dense layer reduces the output dimensionality to the number of target classes. Specifically, it performs a 6×64 **matrix-vector multiplication**, where each of the 6 neurons corresponds to a distinct EMG gesture class. Only **6 MAC units** are required, each operating in parallel to compute the dot product between its corresponding weight row and the input feature vector. The

result is a 6×1 **output vector** that contains raw class scores (logits) representing the activation level of each class.

- **HardMax Function:** In place of using traditional SoftMax for generating output, which computes normalized probability distributions, a more hardware-friendly **HardMax function** is employed. Hardmax identifies the index of the maximum value within the 6×1 vector and outputs a **one-hot encoded vector** based absolute value. This binary vector has a single high bit indicating the predicted class. The use of HardMax eliminates the need for exponential, division, or normalization operations, reducing computational complexity and hardware usage.
- **Output:** The final output is a single integer value or one-hot vector indicating the predicted EMG gesture class. This result can be directly interfaced with control logic to drive actuators, display predictions, or trigger specific application behaviors in real time.

Architecture designed for Model 1 is particularly well-suited for edge computing environments like wearable or prosthetic control units, where low latency and power efficiency are crucial. The modular nature of the design also allows for future expansion, such as the integration of more classes, making it compatible with floating point, adding some more layers to redefine accuracy. The the lightweight and efficient design of the CNN ensures that it can operate within the constrained computational and power budgets typical of battery operated edge devices. This makes it ideal for real-time applications where responsiveness is critical. Moreover, the flexibility of the architecture supports easy deployment across different hardware platforms.

Data Flow for Model 1

The inference process in Model 1 follows a structured pipeline of feature extraction and classification. The key stages are outlined below:

- **Input Preprocessing:** The input to the model is preprocessed electromyography (EMG) test data, organized as a single channel time-series signal having dimension . This data is normalized and segmented appropriately to ensure consistency with the training configuration.

- **Initial Convolutional Layer:** The preprocessed input having dimension (1×6) , which simplifies six time stamps data from single channel, is first passed through a 1D convolutional layer. This layer applies a series of learned filters (kernels 1×2), which were optimized during the training phase, to extract localized temporal patterns across all EMG channels.
- **Non-Linearity via Activation:** The output of the convolutional layer is passed through a **Rectified Linear Unit (ReLU)** activation function. This introduces non-linearity into the model by retaining positive values and zeroing out negative ones, thereby enhancing the model's ability to learn complex patterns.
- **Temporal Downsampling via Pooling:** The activated feature maps are then subjected to temporal downsampling (pool size is 2) using 1D **MaxPooling** layers. This reduces the temporal resolution while preserving the most salient features, improving computational efficiency and model robustness.
- **Hierarchical Feature Extraction:** The combination of convolution, ReLU activation, and pooling is repeated across multiple layers. This deepens the model's capacity to extract hierarchical and increasingly abstract features from the EMG signal.
- **Flattening Feature Maps:** After the final convolutional and pooling operations, the multidimensional feature maps are flattened into a single one-dimensional vector, serving as a compact representation of the input signal.
- **Fully Connected Layers:** The flattened vector is then processed by one or more fully connected (dense) layers. These layers integrate the extracted features and map them to higher-level representations that are more discriminative for classification.
- **Classification via HardMax:** The final dense layer outputs a set of activation values (logits) corresponding to each gesture class. A **HardMax** activation function is applied to select the class with the highest activation, resulting in a one-hot encoded prediction vector.
- **Prediction Evaluation:** The predicted class label (corresponding to the index of the maximum logit) is compared against the ground truth label in the test set to evaluate the model's inference accuracy.
- **Inference-Only Execution:** During deployment, only the forward pass is executed. There is no backpropagation or weight update involved, enabling low-latency and

efficient real-time gesture classification.

5.1.2 Architecture of conv1D Layer1(Model1)

Architecture of Layer 1

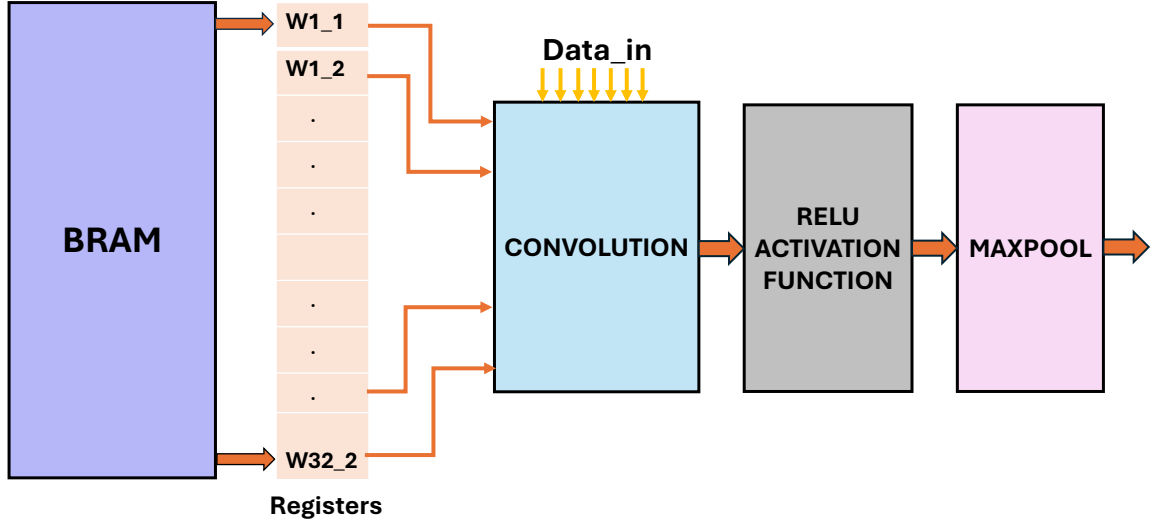


Figure 5-2: Block diagram of architecture of conv1D layer1(Model1).

- **Overview:** Layer 1, shown in Figure 5-2, initiates the feature extraction process from the preprocessed 1D time-series EMG input signal. It consists of a sequence of operations—1D convolution, non-linear activation using ReLU, temporal down-sampling via max pooling, and buffered storage through a shift register. These operations on EMG signals transform them into a compact and informative set of features so that it can be used for subsequent processing in deeper layers.
- **Convolution (Conv1D):** The input EMG signal from a single channel (1D input) is first processed by a **1D convolutional unit** comprising **32 filters**, each having a **kernel size of 2**. In this process, each kernel slides across the input sequence, performing localized dot products with weights in the kernel and then taking the weighted sum to extract temporal features, depending upon the predefined stride (in our case, the stride is 1). The use of multiple filters allows the layer to learn diverse patterns such as signal peaks, which are critical for identifying the underlying muscular activity.

- **Non-linear Activation (ReLU):** The feature maps generated by the conv1D unit are passed through the **Rectified Linear Unit (ReLU)** activation function. This operation is done mainly to introduce nonlinearity into the system, which enables the network to extract complex feature and suppress irrelevant noise in the EMG data, which basically involves making all negative values zero and retaining only the positive activations.
- **Dimensionality Reduction (MaxPooling1D):** To reduce computation in further deep layers, a **max pooling layer** with a **pool size of 2** is applied to the ReLU outputs. This layer slides across the feature maps depending upon pool size and selects the maximum value within each window, thereby halving the actual temporal length in our case. This step enhances computational efficiency by reducing the dimensionality of the feature map .
- **Feature Buffering (Shift Register):** The output of the max pooling operation is stored in a **shift register** structure for **64 clock cycles** because weight loading in conv2D Layer 2 takes time. This temporary data retention enables seamless sequential data access in the next convolution layer. A handshaking mechanism using a **a local acknowledgement signal rest signal** coordinates the flow of data between each layer, ensuring synchronized processing and efficient memory utilization.
- **Efficient Reuse (Time-Multiplexing):** Instead of employing 64 parallel convolution units in the next layer, the buffered data is fed **sequentially** into a **single reusable convolution unit** in Layer 2. A time-multiplexing approach is adopted to use the same hardware across all 64 filters, significantly reducing area, power consumption, and design complexity. This reuse is done using shift registers and multiplexers to dynamically switch contexts for each filter during processing.
- **Summary:** Through a carefully integrated combination of operation such as convolution, activation, pooling, buffering strategies, conv1D Layer 1 is made to extract features by considering effective trade-off between performance and resource efficiency. This foundational layer extracts features from EMG signals and also prepares the data in a compact form that is hardware-friendly for deeper network layers.

5.1.3 Architecture of conv1D Layer 2(Model 1)

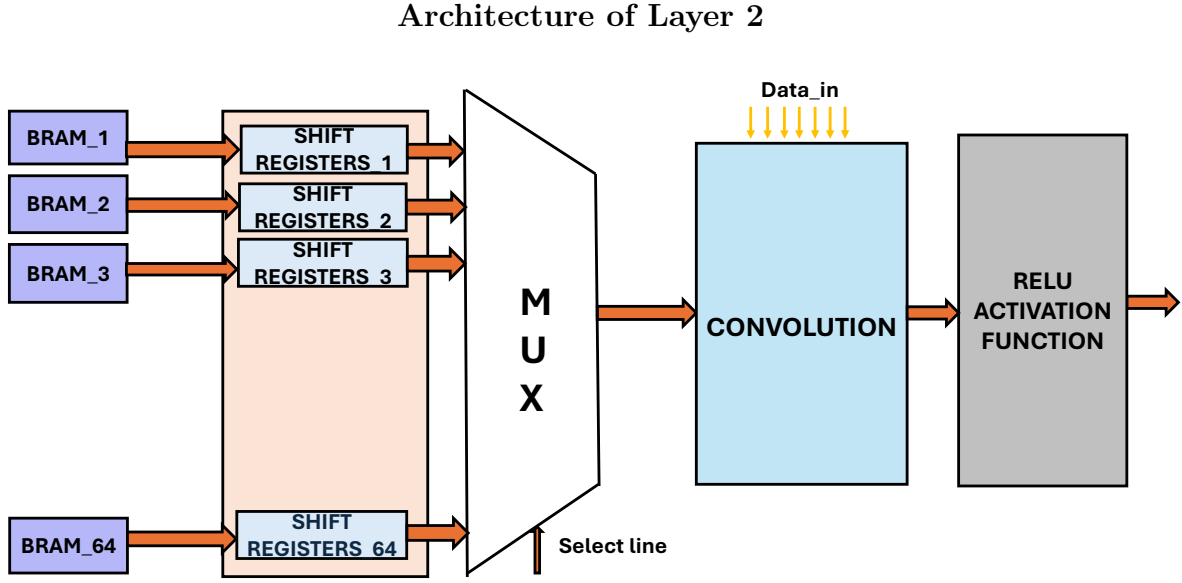


Figure 5-3: Block diagram of architecture of conv1D layer2(Model1).

- **Weight Storage using Block RAMs (BRAMs):** Conv1D Layer 2, as shown in Figure 5-3, is designed to process the output from the first layer using 64 distinct filters, having kernel size of 2×32 . Instead of designing 64 separate convolution units, the design uses a single shared convolution unit hence drastically reducing hardware complexity. To support this, the weights corresponding to each of the 64 filters are stored in separate **Block RAMs (BRAMs)**. This allows the architecture to access the full set of kernel weights for any particular filter with minimal latency and high efficiency. Preloading of weights obtained in BRAMs ensures rapid weight retrieval during real-time operation.
- **Shift Register Interface for Parallel Weight Access:** In layer two, before the convolution operation, weights are transferred from BRAMs into **shift registers**, in this way, shift registers serve as intermediate buffers to simultaneously access all weights in a given kernel. This parallel access of weights is very crucial for maintaining high-speed operation and ensuring that the convolution unit receives a full set of weights for every computation cycle without stalling.
- **Multiplexer (MUX) for Convolutional Unit Reuse:** In order to reduce

hardware complexity, a **multiplexer (MUX)** is employed to select one of the 64 shift registers containing the weights for a specific filter. Now, this selected set of weights is then mapped to the shared convolution unit for further computation. By time-multiplexing the computation, the same hardware can be reused to perform convolution for each filter in sequence. As a result, this approach eliminates the need for 64 separate convolution units, leading to a substantial reduction in power consumption.

- **Convolution and ReLU Activation:** After fetching weights and once the input data is available from previous layers, the shared convolution unit performs 1D convolution, which is basically the dot product between the incoming feature segment and the selected kernel weights, then performing weighted sum . The resulting output then passes through a **Rectified Linear Unit (ReLU)** activation function, which introduces non-linearity into the network.
- **Serial-In Parallel-Out (SIPO) Interface:** The data is fed into **Serial-In Parallel-Out (SIPO)** interface after ReLu activation function. This module collects serialized output from the convolution unit over multiple cycles and converts it into parallel output, because this conversion is essential for feeding data efficiently into the next processing stage that is dense, which operates in parallel. The SIPO structure thus acts as an interface between sequential computation and parallel processing, maintaining throughput while ensuring compatibility between different layers.
- **Optimization Strategy: Hardware Sharing and Efficient Weight Loading:** The main optimization in this layer lies in the **reuse of a single convolution unit** across all 64 filters, facilitated by BRAM-based weight storage ,then storing weights in 64 shift registers and then a MUX-controlled shift register system. Unlike conventional designs that dedicate one unit per filter, this architecture minimizes hardware redundancy and power overhead. Additionally, loading weights from BRAMs into shift registers allows concurrent preparation of data while computations are underway thus improving latency, maximizing throughput and ensuring that the convolution unit does not have to wait for further processing.

5.1.4 Architecture of dense layer

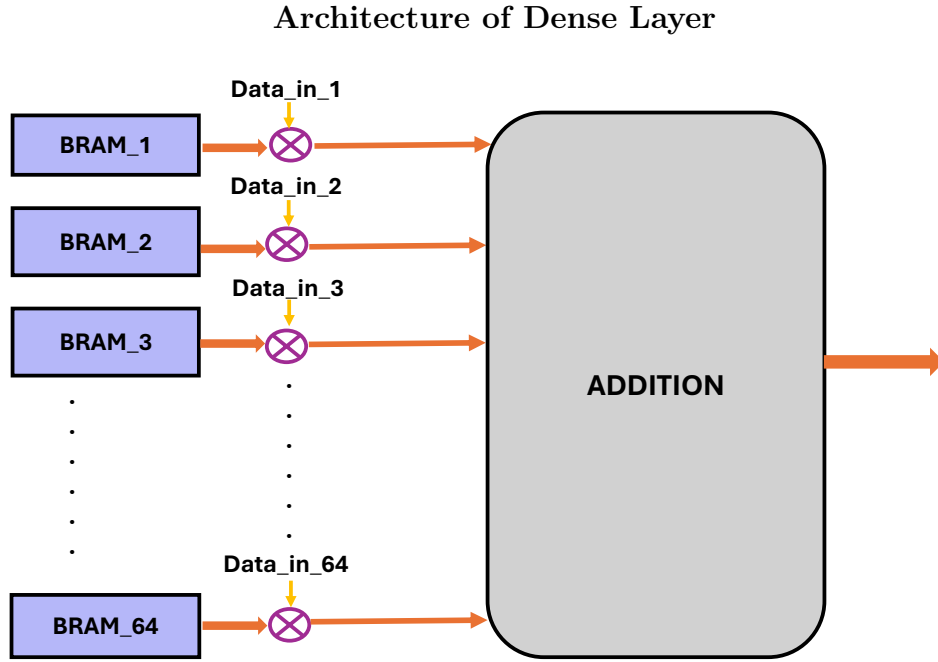


Figure 5-4: Block diagram of architecture of dense layer.

- The dense layer performs a crucial operation in the neural network by performing matrix-vector multiplication between a weight matrix and an input vector, which helps in converging extracted features. The first dense layer as shown in fig.5-4 involves multiplying a 64×64 weight matrix by a 64×1 input vector, resulting in a 64×1 output vector. The input vector, which consists of 64 parallel values, is provided to the system in one clock cycle. The corresponding weight matrix is stored in block RAM (BRAM) as 64 rows, where each row holds the weights linked to one output neuron.
- To speed up computation and improve throughput, the design uses 64 multiply cumulative units (MAC) operating in parallel. Each MAC unit is assigned to one row of the matrix, which performs a weighted sum. The weights for all 64 rows are accessed simultaneously from BRAM, and then each row of input from 64 filters is broadcasted to all MAC units at once through SIPO. As a result, each MAC performs its element-wise multiplication and accumulation, producing a 1D vector output 64×1 . Because all 64 MAC units work together in parallel, the entire output vector is generated in just a few cycles after loading the weights and input data.

This approach ensures minimal latency and supports high-throughput processing, which is essential for real-time inference.

- The second dense layer follows a similar principle but operates on a differently sized matrix. Here, a 6×64 weight matrix is multiplied by the same 64×1 input vector to produce a 6×1 output vector. The input vector is again loaded in parallel, and the weight matrix is stored in BRAM with 6 rows representing the weights for six output neurons. To match this setup, 6 MAC units run concurrently, each handling one row of weights. These MACs fetch their weights simultaneously from BRAM and perform the dot product with the broadcast input vector to generate the output. The entire output vector of six values is therefore calculated in a few clock cycles, just as in the first dense layer. The outputs from this final dense layer are passed through an activation function such as ReLU to introduce non-linearity, which helps the network learn more complex patterns. This structure is well-suited for classification tasks where the number of output classes is relatively small—in this case, six.
- **Summary:** It can be concluded the architecture balances parallelism and resource usage by assigning a dedicated MAC unit to each output neuron, enabling fast matrix-vector multiplication without the need for iterative or sequential processing. By storing weights in BRAM and broadcasting inputs, it reduces memory latency and efficiently manages data flow. This design makes the dense layers highly efficient and capable of supporting real-time processing requirements in hardware-accelerated neural networks.

Table 5.1: Summary of CNN Architecture and data flow in Model 1

Stage	Description
Input	Real-time preprocessed EMG signals with 32 channels, streamed into the hardware system.
Conv1D Layer 1	32 filters with kernel size 2 are applied. Convolution is performed using MAC units and shift registers. ReLU activation is applied to introduce non-linearity.
MaxPooling1D Layer 1	Pooling operation with pool size 2 reduces the temporal resolution by half while preserving key features.
Conv1D Layer 2	64 filters applied sequentially using a single shared convolution unit. Weight selection is controlled via multiplexers. ReLU activation follows.
Adders Block	Performs local summation of feature maps to assist with feature aggregation before dense layers.
Flatten + SIPO	Converts sequential convolution outputs into a parallel 1D vector using Serial-In Parallel-Out register structure.
Dense Layer 1	Fully connected layer with 64 neurons. Computation is parallelized using 64 MAC units. ReLU activation is applied.
Dense Layer 2	Fully connected layer with 6 output neurons corresponding to gesture classes. Computation done via 6 MAC units.
HardMax Function	Selects the index of the maximum activation value and outputs a one-hot encoded vector representing the predicted gesture class.
Output	One-hot encoded vector or class index representing the predicted EMG gesture, directly usable for real-time decision-making or actuation.

5.2 Implementation of Model 2

5.2.1 Architecture of Model 2 CNN

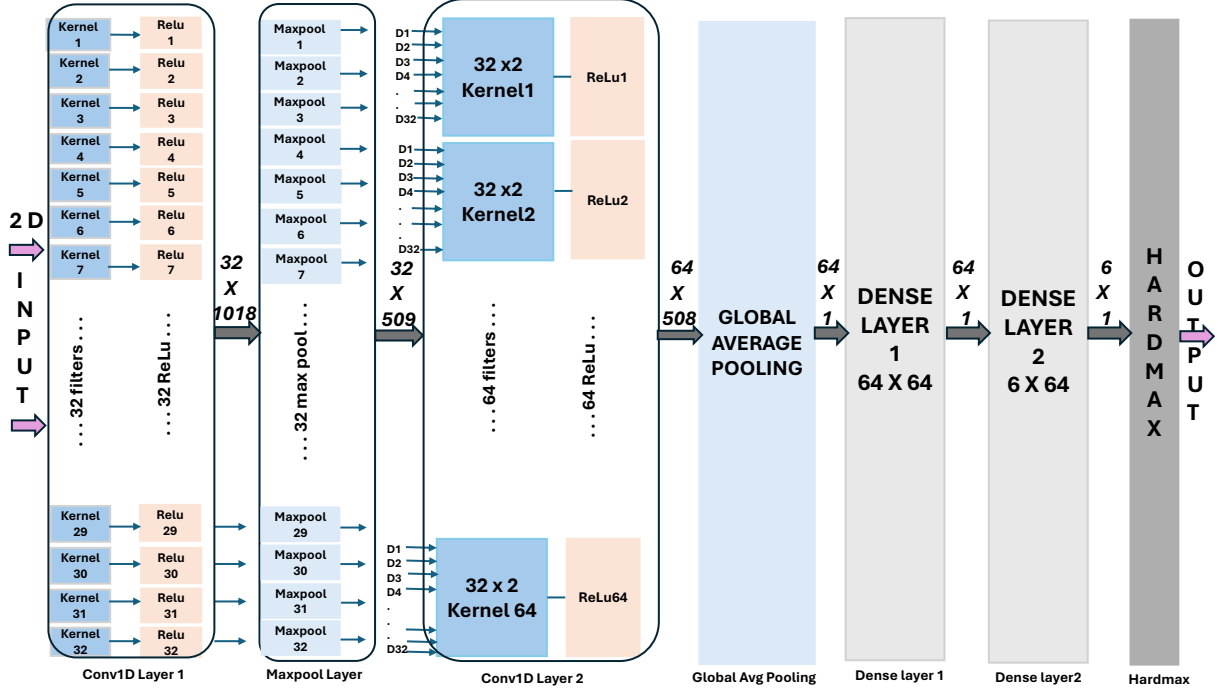


Figure 5-5: Block diagram of Architecture of model 2

Model 2 involves model training using multi-channel EMG signals, with gesture recognition performed by a hardware classifier during the testing phase. The hardware classifier comprises a CNN architecture as shown in figure5-5 designed specifically to process multi-channel input, in our case there are 6 channels. We have implemented the inference phase of this CNN architecture using Verilog HDL, incorporating the learned parameters obtained from training phase, for low latency real-time gesture recognition. The detailed specifications of each layer in the proposed CNN architecture Model 2, tailored for multi-channel EMG signal processing, are provided below:

- **Conv1D Layer1:** 32 filters, (2x6) kernel, ReLU
- **MaxPooling1D:** Pool size 2

- **Conv1D Layer2:** 64 filters, (32×2) kernel, ReLU
- **Global Avg Pooling:** $(64 \times 508) \rightarrow (64 \times 1)$
- **Dense1:** 64 units
- **Dense2:** 6 units
- **Output:** Hardmax

Input Representation

The input to Model 2 is a structured multidimensional time-series signal with 6 independent channels, each containing 1019 time stamps. This forms an input tensor of shape 1019×6 , analogous to a small image where each "column" is a sensor and each "row" is a point in time. This structured input allows the model to treat time and inter-channel information with equal importance, enabling early layers to extract short-term temporal signals and cross-channel patterns.

Conv1D Layer 1

The first layer is designed to extract local features for which it uses 32 convolutional filters having kernel size (2×6) and stride(1) and, each spanning:

- **Two adjacent time steps**, to extract local temporal features
- **All six channels**, to recognize cross-sensor correlations

This layer not just extracts patterns from a single signal but actively integrates signal across different sensors. **Activation and Pooling** After these filters extracts a feature map, the ReLU (Rectified Linear Unit) activation introduces non-linearity, ensuring the model can deal with complex features. This is followed by a max pooling operation which, reduces temporal resolution by

half when pool size is two. The result is a set of abstracted local features, compressed in time, and prepared for higher-level computation and thesis.

Conv1D Layer 2

Layer 2 takes the 32 feature maps from the previous stage and processes them using 64 deeper filters (kernel size 2×32). Each filter now:

- Observes 2-time steps
- Operates across all 32 feature maps

This layer enables the model not just to recognize low-level patterns (e.g., signal spikes and peaks), but also to learn compositions of those patterns and temporal structures across sensor modalities. **Direct Dataflow (Streaming)** Crucially, Layer 2 operates in a streaming mode:

- No intermediate buffering is used
- Data flows directly from Layer 1's output to Layer 2's filters using local reset handshaking

This pipelined architecture reduces memory latency and suits real-time systems. Each new activation is generated and consumed instantly, reflecting a continuous inference pipeline.

Global Average Pooling (GAP)

At this stage, the output is set of 64 feature maps, each of length 508 (time stamps). However, this high temporal dimension representation is unsuitable for feeding into dense layer and classification directly. This GAP layer basically bridge the gap between temporal features and class-level understanding.

Conceptual Role Instead of looking at precise locations of features, GAP summarizes the presence of each feature across time:

- It computes the mean activation of each feature map.
- This process reflects how many times this feature was present across temporal sequence.

The result is a compact 64 x 1 vector, one value per feature, which extracts away the information when a pattern occurred and focuses on whether it occurred.

Fully Connected Layers

Dense Layer 1: Feature Interaction This layer performs weighted sum i.e. matrix multiplication the 64-dimensional pooled vector into a new 64-dimensional vector via a dense matrix multiplication. Unlike convolutional layers that work with local context, this layer allows:

- Global interaction between all extracted features
- Recombination and convergence of features into higher-order semantic representations

A dense layer is called "fully connected" because every neuron receives input from all neurons in the previous layer, which allows it to combine and mix all extracted features, enabling complex classification easier.

Dense Layer 2: Classification Space Mapping

The second dense layer reduces the feature vector from 64 to 6 dimensions—one for each target class. Each output node in this layer corresponds to a class and these outputs are raw scores or "logits" of each class.

Output Layer (Hardmax)

Conceptual Role: The **hardmax layer** has a decision mechanism that selects the class with the highest logit score. Unlike softmax, it does not produce a probability distribution, making it ideal for inference applications where only the most likely class matters—such as real-time gesture classification.

Operation: During inference, hardmax computes the output as:

$$\hat{y} = \arg \max_{i \in \{1, \dots, 6\}} z_i$$

where z_i is the logit value for class i . This eliminates the need for exponential and normalization computations, reducing latency and resource usage in hardware implementations.

Data Flow in CNN Model 2

1. Input Layer

Input Shape: 1019×6

Description: Multichannel time-series EMG signal with 6 independent sensor channels and 1019 time steps per channel.

2. Layer 1 – 1D Convolution

Operation: 32 filters of size 2×6 applied along the temporal dimension, stride = 1, no padding.

Computation: $T_{\text{out}} = 1019 - 2 + 1 = 1018$

Output Shape: 32×1018 (post-ReLU activation)

3. Layer 1 – Max Pooling

Operation: 1D max pooling with window size 2, stride = 2.

Computation: $T_{\text{pooled}} = \lfloor \frac{1018}{2} \rfloor = 509$

Output Shape: 32×509

4. Layer 2 – 1D Convolution

Dataflow Strategy: Streaming mode with pipelined processing, no intermediate BRAM buffering.

Operation: 64 filters of size 2×32 , stride = 1, no padding.

Computation: $T_{\text{out}} = 509 - 2 + 1 = 508$

Output Shape: 64×508 (post-ReLU activation)

5. Global Average Pooling (GAP)

Operation: Compute the mean across the temporal dimension for each feature map.

Formula: $\text{GAP}_i = \frac{1}{508} \sum_{t=1}^{508} x_{i,t}, \quad \text{for } i = 1, \dots, 64$

Output Shape: 64×1 (flattened to 64)

6. Dense Layer 1

Input: 64 **Operation:** Fully connected layer with 64 neurons

Weight Matrix: 64×64 **Output:** 64

7. Dense Layer 2 (Classifier)

Input: 64 **Operation:** Fully connected layer with 6 neurons

Weight Matrix: 6×64 **Output:** 6 (logits)

8. Output Layer

Output: 6-dimensional logit vector

Inference: Hardmax selects the class with highest logit: $\hat{y} = \arg \max_{i \in \{1, \dots, 6\}} z_i$

Training: Logits passed through a softmax followed by cross-entropy loss.

5.2.2 Architecture of conv1D layer 1(Model 2)

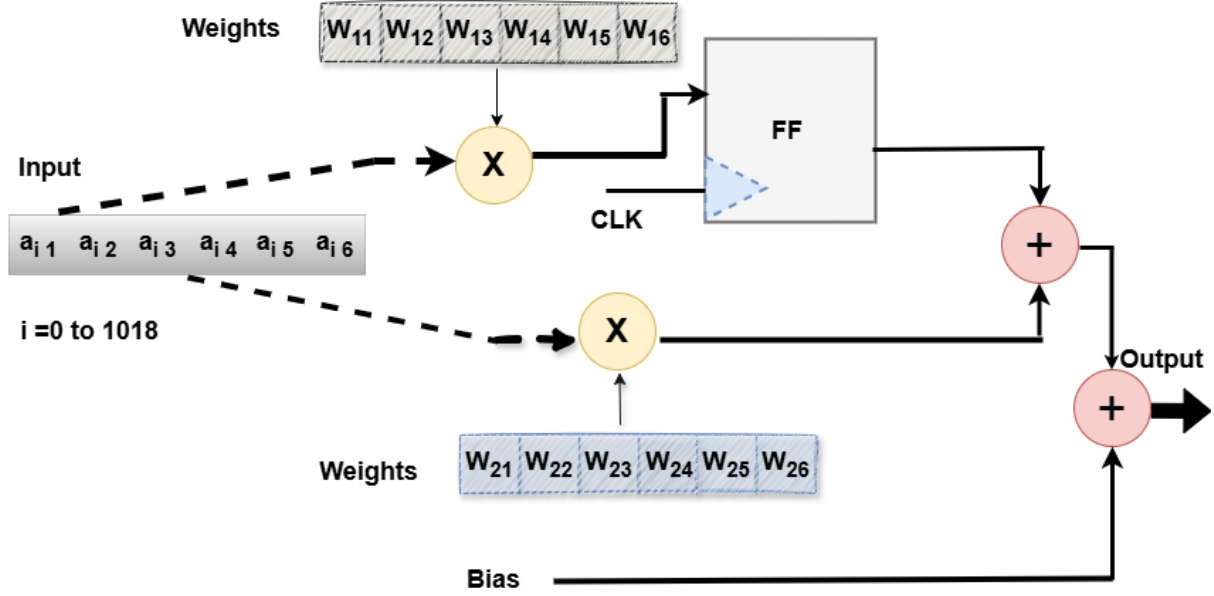


Figure 5-6: Block diagram of Architecture of conv1D layer1(Model 2)

Conv 1D Layer 1 Architecture

The architecture of the convolution unit in **Conv 1D Layer 1**, shown in Figure 6-4, implements a **1D convolution** over a **2D input matrix** of shape 1019×6 , where 1019 represents time steps and 6 represents input channels.

This layer has **32 filters**, each sized 2×6 , which means each filter spans **two consecutive time steps** and covers **all six channels**. During convolution, two rows (time steps) at a time are taken from the input, multiplied element-wise with filter weights, and summed to produce the output. This sliding window moves with stride 1 along the time axis, resulting in **1018 valid positions** since $1019 - 2 + 1 = 1018$.

Each filter consists of $2 \times 6 = 12$ weights, so with 32 filters, there are a total of 384 weights. Adding a bias term per filter brings the total trainable parameters to 416. The output of this layer is a matrix of shape 32×1018 ,

where 1018 is the number of time steps post-convolution, and 32 is the number of filters (features extracted).

As illustrated in Figure 6-4, at time $t = 0$, the first two rows (Row 0 and Row 1) are convolved with each filter, producing the output vector y_0 of size 32. This window then slides one step at a time until the last two rows, generating the final output vector y_{1017} .

Layer 1 Hardware Reuse using Multiplexer (MUX) – Model 2

In **Model 2, Layer 1**, similar to **Model 1**, we implement an efficient hardware reuse strategy for convolution operations using a **multiplexer (MUX)**. Since the filter size is fixed at 32, we can exploit this uniformity to reuse a **single convolution engine** for all filters through time-multiplexing. Instead of instantiating separate convolution units for each filter, we use a MUX to sequentially select the input feature maps and corresponding filter weights. These are routed to a shared convolution hardware block, as illustrated in Figure 6-4.

This MUX-based approach offers several benefits:

- **Reduced Hardware Footprint:** Only one convolution block is needed, significantly minimizing resource usage.
- **Lower Power Consumption:** Sharing logic reduces dynamic power due to fewer active logic blocks.
- **Scalability:** The design can scale with additional filters by extending the MUX control logic, without increasing core convolution resources.
- **Consistency with Model 1:** This technique follows the same time-division reuse principle as in Model 1, ensuring a consistent design

methodology across models.

The trade-off is an increase in processing time, as each filter is processed sequentially. However, for applications where hardware efficiency is prioritized over latency, this design is highly advantageous.

In summary, **Layer 1 performs time-based convolution** where the kernel spans **2 time steps and all input features**, extracting **32 new features per time step** over 1018 time steps. This makes the layer effective for capturing **local temporal patterns** while being parameter-efficient with only 416 trainable parameters.

Computation in Layer 1 Filters

Input Matrix (1019×6):

$x_{0,0}$	$x_{0,1}$	$x_{0,2}$	$x_{0,3}$	$x_{0,4}$	$x_{0,5}$
$x_{1,0}$	$x_{1,1}$	$x_{1,2}$	$x_{1,3}$	$x_{1,4}$	$x_{1,5}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$x_{1018,0}$	$x_{1018,1}$	$x_{1018,2}$	$x_{1018,3}$	$x_{1018,4}$	$x_{1018,5}$

Input matrix X : 1019 time steps $\times$ 6 channels

Filter Structure (2×6):

$w_{11}^{(k)}$	$w_{12}^{(k)}$	$w_{13}^{(k)}$	$w_{14}^{(k)}$	$w_{15}^{(k)}$	$w_{16}^{(k)}$
$w_{21}^{(k)}$	$w_{22}^{(k)}$	$w_{23}^{(k)}$	$w_{24}^{(k)}$	$w_{25}^{(k)}$	$w_{26}^{(k)}$

Single filter k of shape 2×6 (12 parameters)

Output Matrix (32×1018):

$y_0^{(1)}$	$y_1^{(1)}$	$y_2^{(1)}$	$\cdots$	$y_{1016}^{(1)}$	$y_{1017}^{(1)}$
$y_0^{(2)}$	$y_1^{(2)}$	$y_2^{(2)}$	$\cdots$	$y_{1016}^{(2)}$	$y_{1017}^{(2)}$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
$y_0^{(31)}$	$y_1^{(31)}$	$y_2^{(31)}$	$\cdots$	$y_{1016}^{(31)}$	$y_{1017}^{(31)}$
$y_0^{(32)}$	$y_1^{(32)}$	$y_2^{(32)}$	$\cdots$	$y_{1016}^{(32)}$	$y_{1017}^{(32)}$

Output of Layer 1: 32 features $\times$ 1018 time steps

Computation of Output Elements:

For each filter $k \in \{1, 2, \dots, 32\}$ and each time step $t \in \{0, 1, \dots, 1017\}$, the output value $y_{k,t}$ is computed by applying the convolutional kernel over the input patch corresponding to time steps t and $t + 1$, across all input channels. The mathematical formulation is:

$$y_{k,t} = \sum_{i=0}^1 \sum_{j=0}^5 w_{i,j}^{(k)} \cdot x_{t+i,j} + b^{(k)}$$

Where:

- $x_{t+i,j}$ is the input signal at time step $t + i$ and channel j
- $w_{i,j}^{(k)}$ is the convolution weight of filter k at position (i, j)
- $b^{(k)}$ is the bias term associated with filter k

The result is a 2D activation map with dimensions 32×1018 , where each row corresponds to a filter and each column represents a temporal activation at a specific time step.

5.2.3 Architecture of maxpool layer(Model 2)

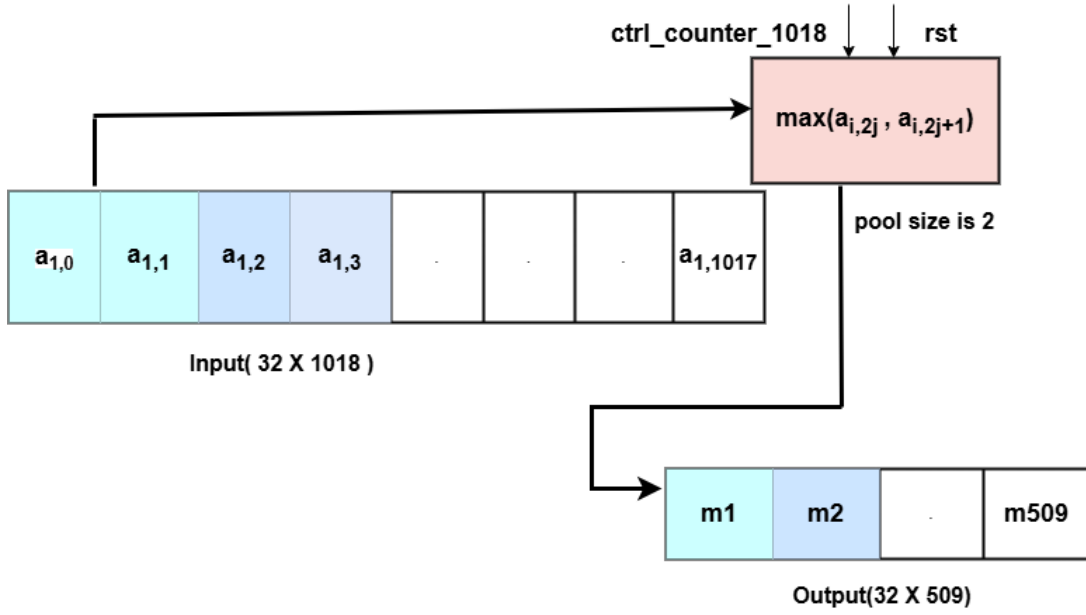


Figure 5-7: Architecture of maxpool unit, pool size 2

Streaming Max Pooling Layer Design

The Max Pooling layer is used to reduce the spatial dimensions of feature maps by selecting the maximum value within a fixed-size window. In this design, the layer processes a stream of **1018 input values**, arriving sequentially, and performs max pooling dynamically as the data flows in.

A window of fixed length, determined by the **pool size**, slides across the input stream. For each window, the system continuously tracks and updates the maximum value seen so far. Rather than storing the entire window in memory, the design compares values **on-the-fly**, enabling efficient use of hardware resources.

To implement this in hardware we followed the architecture illustrated in figure. 6-3, we use a single comparator unit, which is reused for all pooling windows. The control logic is responsible for determining when comparisons

should occur and when the system should reset to begin evaluating the next window.

The pooling mechanism is governed by the following components:

- A **counter** that increments with each incoming input value and keeps track of how many values have been compared in the current window.
- A **control signal**, driven by the counter, that enables the comparator to update the stored maximum value if the current input is greater.
- A **local reset signal**, triggered once the window is complete (i.e., the counter reaches the pool size), which resets the current maximum value and prepares the logic to process the next window.
- The **stride**, which determines how the pooling window moves, is managed by updating the local reset timing. This ensures the comparison window shifts appropriately with respect to the stride setting.

This architecture enables the reuse of a single hardware unit for all max pooling operations, significantly reducing resource usage such as LUTs and flip-flops. It also supports continuous streaming input, making it well-suited for real-time processing scenarios where latency and area are critical.

The first 1D convolutional layer produces output feature matrix which has a shape of 32×1018 , where 32 corresponds to the number of filters applied, and 1018 corresponds to the number of time steps after convolution with a 2×6 kernel.

To reduce the temporal dimensionality and retain the most significant feature map, a **1D max pooling** operation is performed along the time axis (i.e., across the columns). This pooling uses Pool size is ,Stride is 2.

The pooling operation takes every two consecutive time steps for each feature and computes their maximum. Formally, for feature channel k and time index t , the max pooling output is:

$$m_k^{(t)} = \max(y_k^{(2t)}, y_k^{(2t+1)})$$

This reduces the temporal length from 1018 to 509 (i.e., $1018 \div 2 = 509$), while retaining all 32 feature dimensions. Therefore, the output shape after max pooling becomes 32×509 , which significantly reduces computational load for the next layers.

Computation of Maxpool layer

$y_0^{(0)}$	$y_0^{(1)}$	$y_0^{(2)}$	$\dots$	$y_0^{(1017)}$
$y_1^{(0)}$	$y_1^{(1)}$	$y_1^{(2)}$	$\dots$	$y_1^{(1017)}$
$y_2^{(0)}$	$y_2^{(1)}$	$y_2^{(2)}$	$\dots$	$y_2^{(1017)}$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
$y_{31}^{(0)}$	$y_{31}^{(1)}$	$y_{31}^{(2)}$	$\dots$	$y_{31}^{(1017)}$

Output of Layer 1: 32 features $\times$ 1018 time steps

Max Pooling Computation:

For every pair of columns $(y_k^{(2t)}, y_k^{(2t+1)})$, compute the maximum:

$$m_k^{(t)} = \max(y_k^{(2t)}, y_k^{(2t+1)}), \quad t = 0, 1, \dots, 508, \quad k = 0, 1, \dots, 31$$

Output After Max Pooling (32×509)

$m_0^{(0)}$	$m_0^{(1)}$	$m_0^{(2)}$	$\dots$	$m_0^{(508)}$
$m_1^{(0)}$	$m_1^{(1)}$	$m_1^{(2)}$	$\dots$	$m_1^{(508)}$
$m_2^{(0)}$	$m_2^{(1)}$	$m_2^{(2)}$	$\dots$	$m_2^{(508)}$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
$m_{31}^{(0)}$	$m_{31}^{(1)}$	$m_{31}^{(2)}$	$\dots$	$m_{31}^{(508)}$

After Max Pooling: 32 features $\times$ 509 time steps

This pooled output serves as the input to **Layer 2**, which uses 64 filters for further feature extraction.

5.2.4 Architecture of conv1D layer2(Model 2)

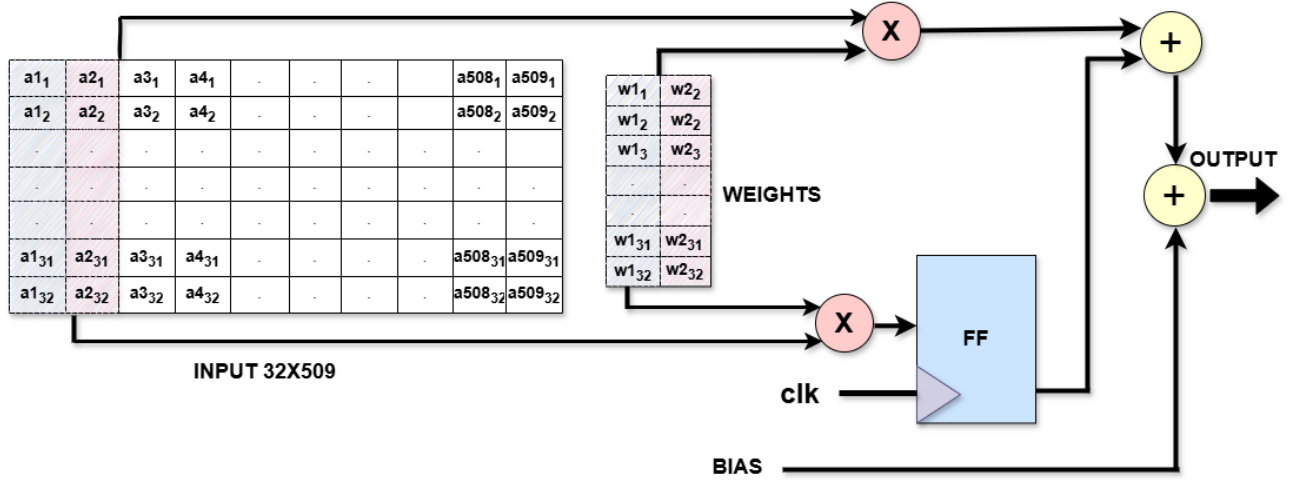


Figure 5-8: Block diagram of Architecture of layer 2(Model2)

The architecture of Layer 2, as illustrated in Figure 6-6, implements a 1D convolution over a 2D input matrix of shape 32×509 , where 509 represents time steps and 32 represents the number of input channels (i.e., output features from Layer 1). The layer applies 64 filters, each of size 2×32 , which means each filter spans two consecutive time steps and covers all 32 channels. During convolution, the layer processes two columns at a time (i.e., two time

steps) from the input and computes the dot product with each of the 64 filters. This is done in a sliding manner along the time axis with a stride of 1, resulting in $509 - 2 + 1 = 508$ valid positions.

Each filter contains $2 \times 32 = 64$ weights. Since there are 64 such filters, the total number of weight parameters is:

$$64 \times 64 = 4096$$

Each filter also has a bias term, adding 64 more parameters. Hence, the total number of trainable parameters in Layer 2 is:

$$4096 + 64 = 4160$$

The output of this convolutional layer is a matrix of shape 64×508 , where 508 corresponds to the number of time steps after convolution, and 64 corresponds to the number of filters (i.e., the features extracted at each time step).

As shown in the figure 5-8, at time $t = 0$, the first two columns (column 0 and column 1) are multiplied element-wise with the filter weights, summed, and passed through an activation function to produce the first output vector y_0 of size 64. At $t = 1$, the window slides down by one row to cover column 1 and column 2, generating the next output y_1 , and so on until the final pair of rows (column 507 and column 508) is processed to yield the final output vector y_{507} .

Layer 2 performs 1D convolution on the output of the max-pooled Layer 1, where the input is a matrix of size 32×509 , representing 509 time steps and 32 feature channels. A total of 64 filters, each of size 32×2 , with stride 1 is used to perform convolution producing an output of size 508×64 . The

convolution is performed across the time dimension, where each filter slides over two consecutive time steps at a time and computes one scalar output.

Layer 2 Hardware Optimization – Model 2

In **Model 2, Layer 2**, an attempt was initially made to fully optimize the convolution operation by instantiating separate convolution modules for all 64 filters. While this approach enables full parallelism and minimal computation latency, it quickly became evident that the resource requirements were impractical for most hardware targets. Specifically, the combination of a large kernel size and a high number of filters resulted in an excessive consumption of **Look-Up Tables (LUTs)** and **Digital Signal Processing (DSP)** blocks, surpassing the limits of many FPGA platforms.

To strike a balance between performance and resource utilization, we adopted a more hardware-efficient strategy. Instead of creating 64 dedicated convolution units, the filter set was divided into **four groups of 16 filters**. We instantiated only **four convolution modules**, each responsible for processing 16 filters. These units are **time-multiplexed** to sequentially process all 64 filters over multiple clock cycles.

This revised approach introduces a trade-off:

- **Advantages:** Significantly reduced LUT and DSP usage, making the design suitable for mid-range FPGAs and enabling further integration of other system components.
- **Trade-off:** Increased latency due to sequential execution of filter groups over four time slots, as opposed to full parallel execution.

Overall, this compromise between area and throughput allows for a scalable

and resource-conscious hardware implementation, without severely impacting the model’s inference performance.

Computation in Layer 2 filters

Input Matrix (32×509):

$x_{0,0}$	$x_{0,1}$	$x_{0,2}$	$\cdots$	$x_{0,254}$	$x_{0,255}$	$\cdots$	$x_{0,508}$
$x_{1,0}$	$x_{1,1}$	$x_{1,2}$	$\cdots$	$x_{1,254}$	$x_{1,255}$	$\cdots$	$x_{1,508}$
$x_{2,0}$	$x_{2,1}$	$x_{2,2}$	$\cdots$	$x_{2,254}$	$x_{2,255}$	$\cdots$	$x_{2,508}$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
$x_{30,0}$	$x_{30,1}$	$x_{30,2}$	$\cdots$	$x_{30,254}$	$x_{30,255}$	$\cdots$	$x_{30,508}$
$x_{31,0}$	$x_{31,1}$	$x_{31,2}$	$\cdots$	$x_{31,254}$	$x_{31,255}$	$\cdots$	$x_{31,508}$

Input Matrix: 32 Channels $\times$ 509 Time Steps (Expanded View)

Filter Matrix (32×2) for Filter k :

Each of the 64 filters is shaped 32×2 , covering two consecutive time steps and all channels. The filter also has an associated bias $b^{(k)}$.

Output Matrix (32×508):

The output has 32 feature maps (rows), each spanning 508 time steps (columns). This corresponds to the application of filters producing outputs over the valid sliding windows on the input.

Computation of output elements:

For each time step $t \in [0, 507]$ and filter $k \in [0, 63]$, the output is computed

$w_{0,0}^{(k)}$	$w_{1,0}^{(k)}$
$w_{0,1}^{(k)}$	$w_{1,1}^{(k)}$
$w_{0,2}^{(k)}$	$w_{1,2}^{(k)}$
$\vdots$	$\vdots$
$w_{0,15}^{(k)}$	$w_{1,15}^{(k)}$
$\vdots$	$\vdots$
$w_{0,31}^{(k)}$	$w_{1,31}^{(k)}$

Filter k : 32 Channels $\times$ 2 Time Steps

$y_{0,0}$	$y_{0,1}$	$\cdots$	$y_{0,253}$	$\cdots$	$y_{0,507}$
$y_{1,0}$	$y_{1,1}$	$\cdots$	$y_{1,253}$	$\cdots$	$y_{1,507}$
$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\ddots$	$\vdots$
$y_{15,0}$	$y_{15,1}$	$\cdots$	$y_{15,253}$	$\cdots$	$y_{15,507}$
$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\ddots$	$\vdots$
$y_{31,0}$	$y_{31,1}$	$\cdots$	$y_{31,253}$	$\cdots$	$y_{31,507}$

Output: 32 feature maps $\times$ 508 time steps

as:

$$y_t^{(k)} = \sum_{i=0}^1 \sum_{j=0}^{31} x_{t+i,j} \cdot w_{i,j}^{(k)} + b^{(k)}$$

Where:

- $x_{t+i,j}$ is the input at time $t+i$ and channel j
- $w_{i,j}^{(k)}$ is the weight of the k^{th} filter at position (i,j)
- $b^{(k)}$ is the bias term for the k^{th} filter

This equation performs an element-wise multiplication between the input window and the filter, sums the result, and adds the bias, producing one scalar output per filter per valid time window.

5.2.5 Architecture of Global Average Pooling (GAP)

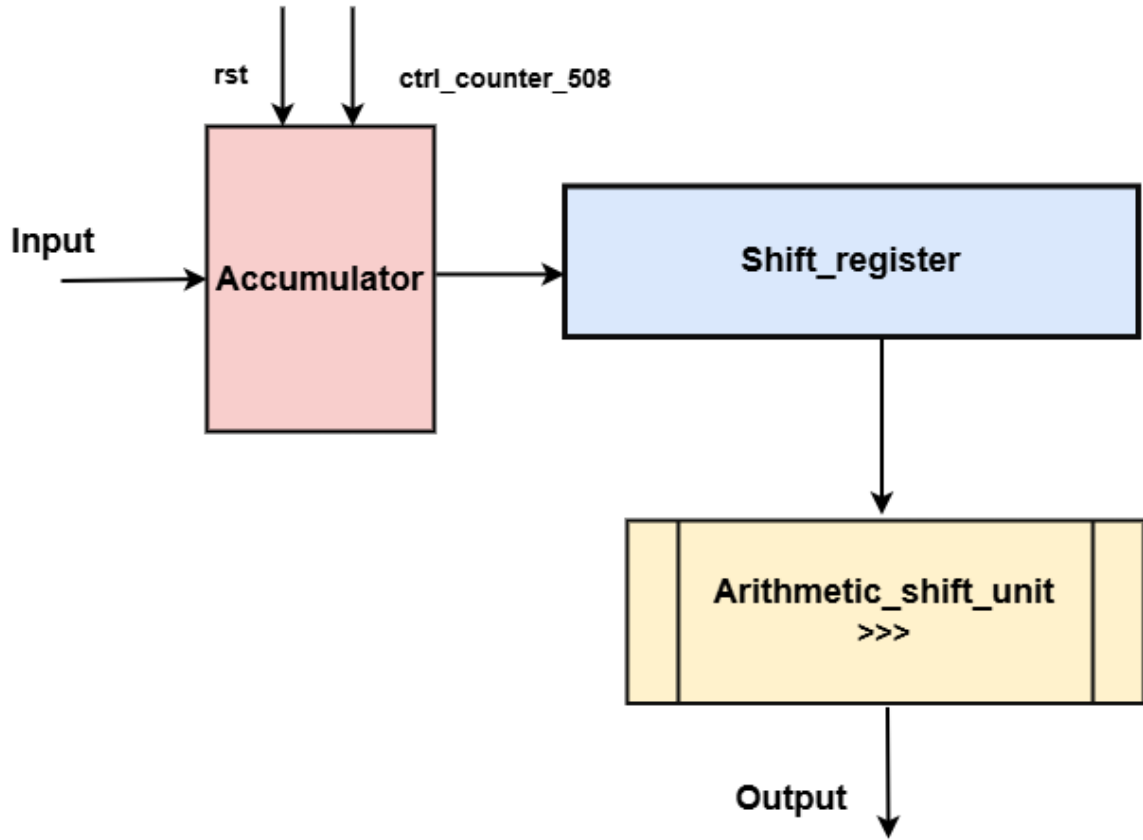


Figure 5-9: Block diagram of Architecture of global average pooling

Global Average Pooling (GAP) is a pooling technique used in CNN to reduce the temporal dimensions of the feature maps. GAP computes the average value across the entire temporal dimension of the feature map, other than selecting the maximum or average value over a small region in case of other pooling.

Global Average Pooling Layer Architecture

In this architecture shown in figure. 5-9, a **Global Average Pooling (GAP)** layer is implemented to reduce the spatial dimension of feature maps from 64×508 to 64×1 . This dimensionality reduction is essential because directly feeding 508 values per channel into a dense layer would result in excessive

computational complexity and resource usage.

To compute the average of the 508 values for each of the 64 channels, a time- and resource-efficient hardware architecture is designed based on an **accumulator-based strategy**.

The core steps of the architecture are as follows:

- **Input Streaming:** The GAP module receives data from 64 channels, each sending one input per cycle. Over 508 cycles, each channel sends 508 values.
- **Accumulator Block:** A single accumulator per channel is used to compute the sum of the 508 inputs. The accumulation is controlled by a **counter-driven control signal**, which enables the accumulator for exactly 508 cycles. During each cycle, the incoming input is added to the current value stored in the accumulator.
- **Shift Register Buffering:** After accumulation is complete, the sum is temporarily stored in a **shift register**. This prevents data loss during pipeline shifting and allows the accumulation process to continue for subsequent batches or channels without overwriting existing data.
- **Arithmetic Right Shift:** Once the full sum is obtained, an **arithmetic right shift** operation is performed on the accumulator output to divide the sum by 508. This is equivalent to computing the average. Arithmetic shifting is used instead of conventional division for its simplicity and efficiency in hardware.
- **Output Generation:** After division, the average value for each channel is output. Thus, the total 64×508 input matrix is reduced to a 64×1

output vector, where each value represents the average of the 508 inputs from the corresponding channel.

This approach offers significant advantages:

- **Resource Efficiency:** Only one accumulator per channel is required, and division is implemented via shifting rather than a complex divider.
- **Latency Management:** Processing occurs over 508 clock cycles, which is acceptable considering the reduced hardware footprint.
- **Compatibility:** The reduced 64×1 output is now feasible to pass into a dense layer, minimizing computation and memory usage.

By combining streaming input, accumulator-based summation, shift-based averaging, and intermediate buffering, the design ensures minimal data loss and efficient execution, making it highly suitable for real-time embedded CNN applications.

Computation in Global Average Pooling (GAP)

The Global Average Pooling (GAP) operation is a widely used technique in modern convolutional neural networks (CNNs), particularly in tasks involving sequence modeling, image classification, or time-series analysis. It serves as a bridge between the deep feature extraction layers and the final decision-making layers such as fully connected or classification layers.

Conceptual role of GAP: GAP is designed to reduce each feature map (produced by convolutional filters) into a single scalar by averaging all its spatial or temporal components. Unlike traditional flattening followed by dense layers, GAP dramatically reduces the number of parameters, thereby

lowering the risk of overfitting and improving the generalization ability of the model.

In our architecture, the output from the final convolutional layer consists of:

- 64 filters (or channels)
- Each filter outputs a 1D feature vector of length 508

This results in a total of 64×508 activation values at the end of the last convolutional stage. The feature vector corresponding to each filter f is:

$$[L_{f,1}, L_{f,2}, L_{f,3}, \dots, L_{f,508}]$$

Here:

- $f \in [0, 63]$ is the filter index
- $i \in [1, 508]$ is the temporal or spatial index
- $L_{f,i}$ represents the activation value at index i for filter f

GAP Operation: The GAP layer takes the average of all 508 activation values for each filter to generate a single scalar output:

$$q_f = \frac{1}{508} \sum_{i=1}^{508} L_{f,i}$$

This operation is repeated independently for each of the 64 filters, producing an output vector:

$$[q_0, q_1, q_2, \dots, q_{63}]$$

The resulting 64-dimensional vector is a highly compressed representation of the most salient information captured by each filter across the tempo-

ral domain. This vector serves as a compact and effective descriptor for downstream classification or regression tasks.

Advantages of GAP:

- **Parameter-free:** GAP contains no trainable weights, which simplifies the model and reduces risk of overfitting.
- **Dimensionality reduction:** Converts high-dimensional feature maps into a much lower-dimensional vector.
- **Robustness:** Provides a global representation, making the model less sensitive to local variations.
- **Interpretability:** The output vector $[q_0, q_1, \dots, q_{63}]$ can be directly linked to the importance of each feature map.

Comparison with Flatten + Dense: Unlike flattening followed by a dense layer (which dramatically increases the number of parameters), GAP provides a structural regularization that maintains model simplicity while preserving critical information.

Example Representation of GAP Computation:

The table below illustrates the GAP process for a few selected filters:

Filter f	Feature Vector $[L_{f,1}, L_{f,2}, \dots, L_{f,508}]$	Global Average $q_f = \frac{1}{508} \sum_{i=1}^{508} L_{f,i}$
0	$[L_{0,1}, L_{0,2}, \dots, L_{0,508}]$	q_0
1	$[L_{1,1}, L_{1,2}, \dots, L_{1,508}]$	q_1
2	$[L_{2,1}, L_{2,2}, \dots, L_{2,508}]$	q_2
3	$[L_{3,1}, L_{3,2}, \dots, L_{3,508}]$	q_3
60	$[L_{60,1}, L_{60,2}, \dots, L_{60,508}]$	q_{60}
61	$[L_{61,1}, L_{61,2}, \dots, L_{61,508}]$	q_{61}
62	$[L_{62,1}, L_{62,2}, \dots, L_{62,508}]$	q_{62}
63	$[L_{63,1}, L_{63,2}, \dots, L_{63,508}]$	q_{63}

Final Output: The result of the GAP layer is a fixed-length vector:

$$\mathbf{q} = [q_0, q_1, \dots, q_{63}] \in \mathbb{R}^{64}$$

This vector acts as the input to the next stage in the neural network pipeline—typically a fully connected (dense) layer or a classification layer with a softmax or hardmax activation depending on the task.

Conclusion: Global Average Pooling is a principled and efficient way to perform dimensionality reduction while preserving the essence of learned features. Its simplicity, effectiveness, and regularization benefits make it a standard component in modern deep learning architectures.

Table 5.2: Summary of CNN Model 2 Architecture and Data Flow

No.	Layer	Output Shape	Description
1	Input	1019×6	multichannel EMG signal; 6 channels representing sensor inputs over 1019 time steps.
2	Conv1D-1	32×1018	Applies 32 temporal filters (2×6) across channels to extract short-term features. Stride = 1, no padding. ReLU activation follows.
3	MaxPool-1	32×509	Downsamples the temporal resolution via 1D max pooling (window = 2, stride = 2), reducing redundancy.
4	Conv1D-2	64×508	Uses 64 filters (2×32) to capture deeper temporal dependencies. Streaming mode (no BRAM) for pipelined efficiency.
5	Global Avg. Pool	64×1	Aggregates temporal data by computing the mean over all 508 time steps for each feature map.
6	Dense Layer 1	64	Fully connected layer with 64 units to learn abstract high-level features from pooled representations.
7	Dense Layer 2	6	Classifier layer with 6 output neurons, each corresponding to a class.
8	Output Layer	6 logits	Final logits used for prediction. Hardmax for inference; softmax + cross-entropy during training.

Chapter 6

Results

6.1 Simulation output of CNN Model 1

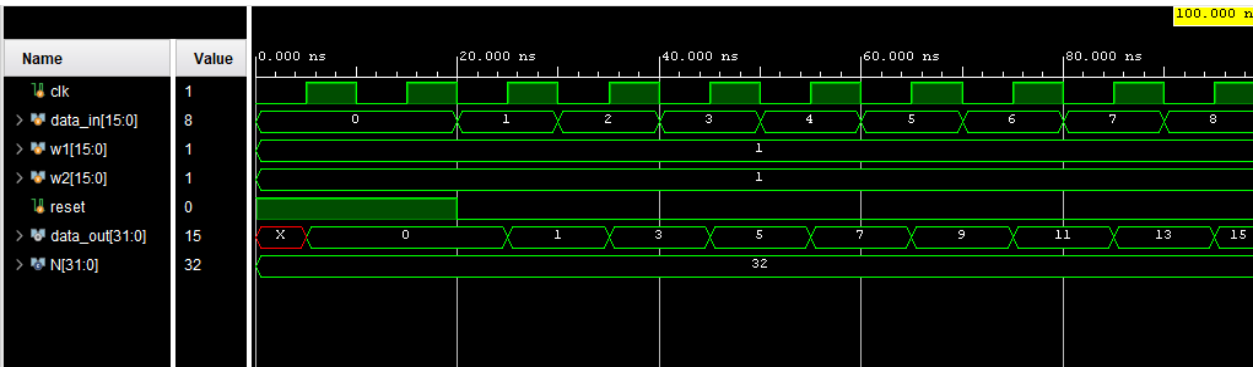


Figure 6-1: Simulation result for conv 1D

Fig. 6-1 illustrates the output of the 1D convolution module performed on the input *data_in* (kernel size is 2, weights w_1 and w_2 are 1), where *data_out* is the output.

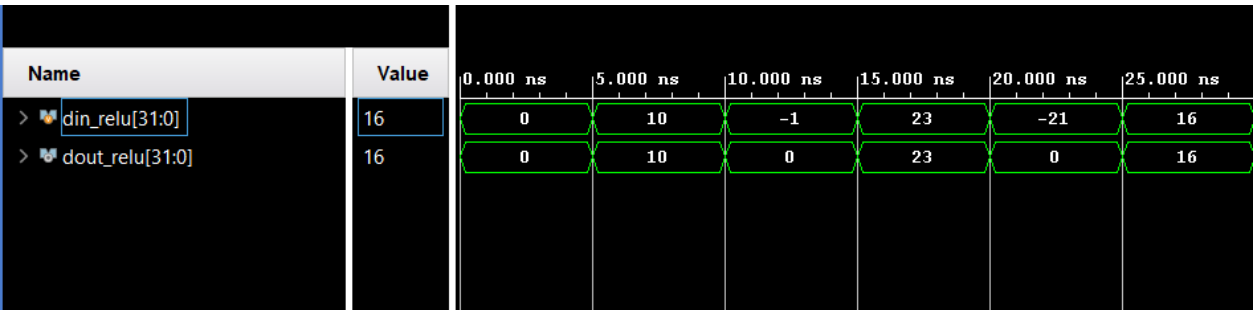


Figure 6-2: Simulation result for ReLU

Fig. 6-2 illustrates the output waveform of the ReLU activation function.

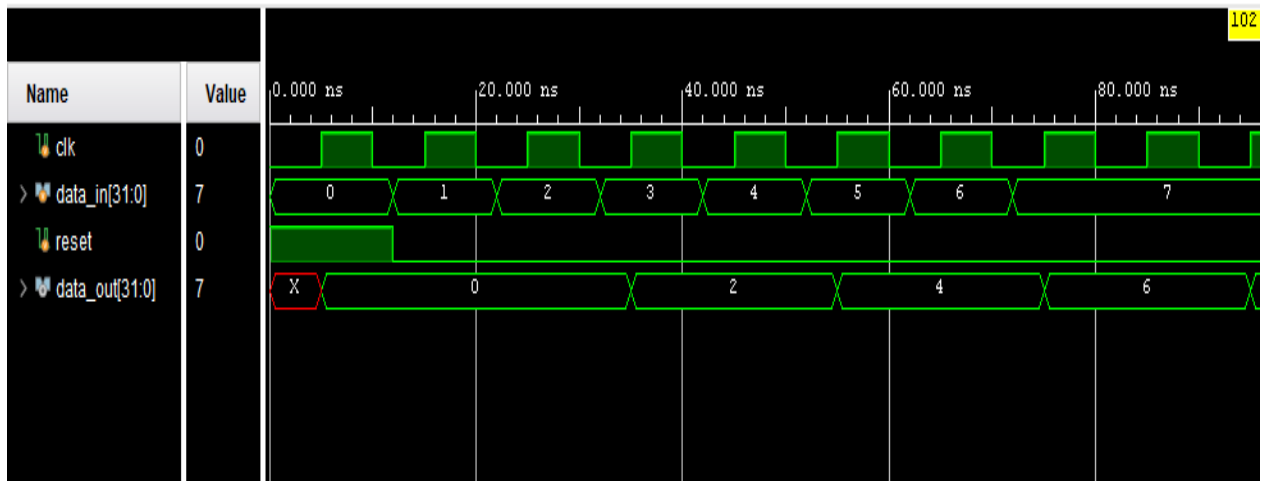
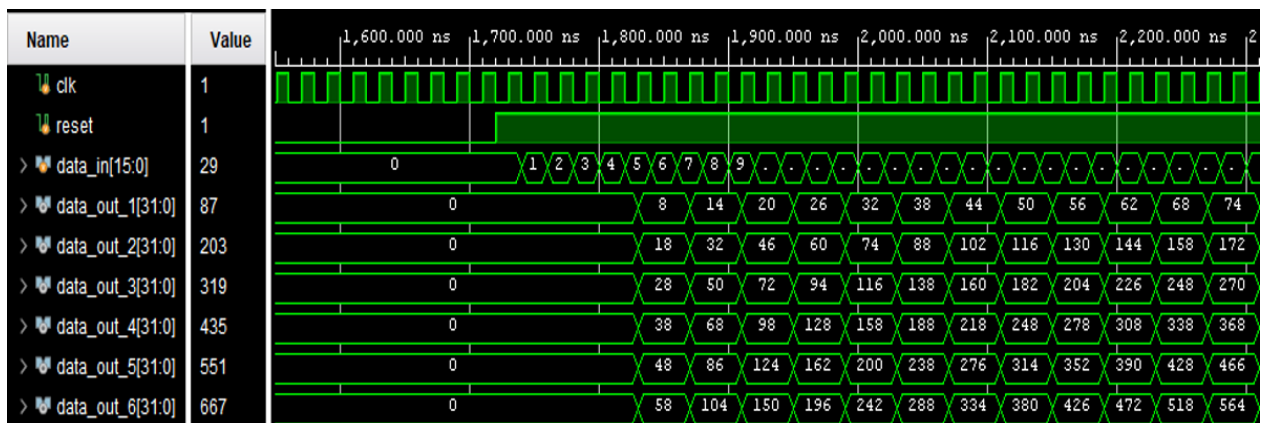


Figure 6-3: Simulation result for MaxPool

Fig. 6-3 displays the result of the max pooling module (pool size is 2), performed on input named *data_in*, and the module has output named *data_out*.



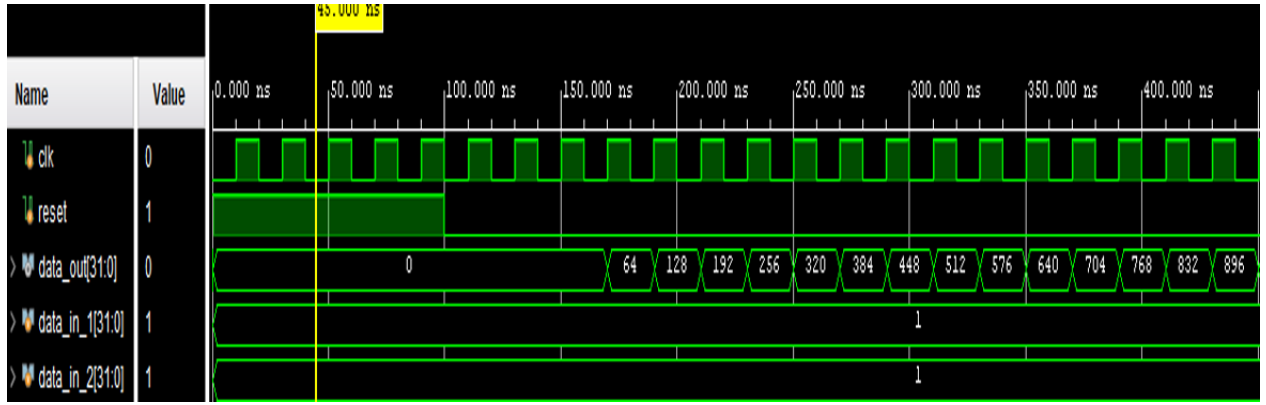


Figure 6-5: Simulation result for Dense Layer 1

Fig. 6-5 shows Dense Layer1 output which receives input from the output of Layer 2. This layer performs matrix multiplication of a 64-dimensional vector (obtained by flattening the output from Layer 2) with a 64×64 weight matrix. For testing, the input vector is initialized with all 1's. The layer utilizes 64 RAMs, each storing a 64-element column of the weight matrix, and each column in the RAMs is initialized with values 1, 2, 3, ... respectively.

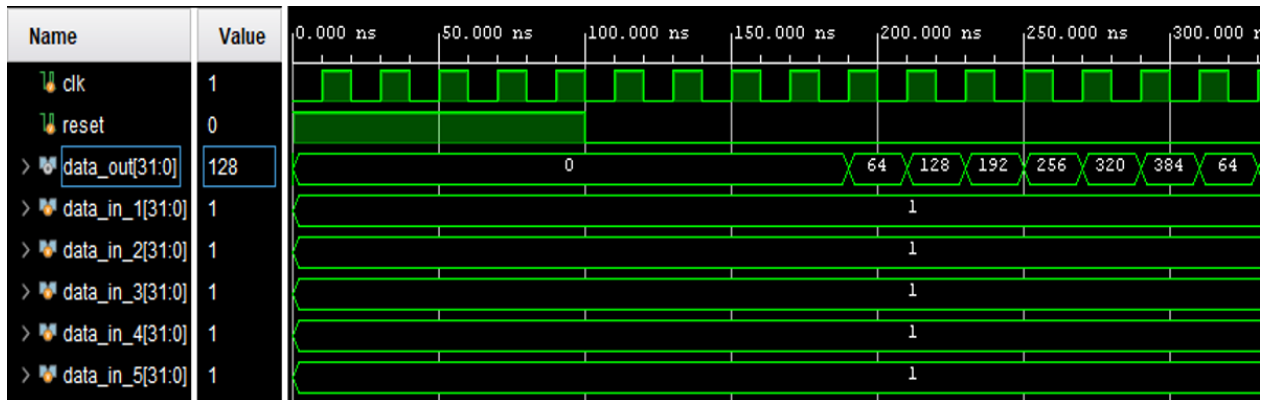


Figure 6-6: Simulation result of Dense Layer 2

Fig. 6-6 displays the output of Dense Layer 2, which receives input from Dense Layer 1. This layer performs matrix multiplication of a 64×1 column vector (obtained from Dense Layer 1) with a 6×64 weight matrix. For testing, the input vector is initialized with all 1's. The layer utilizes 6 RAMs, each storing a 64-element column of the weight matrix, and each column in the

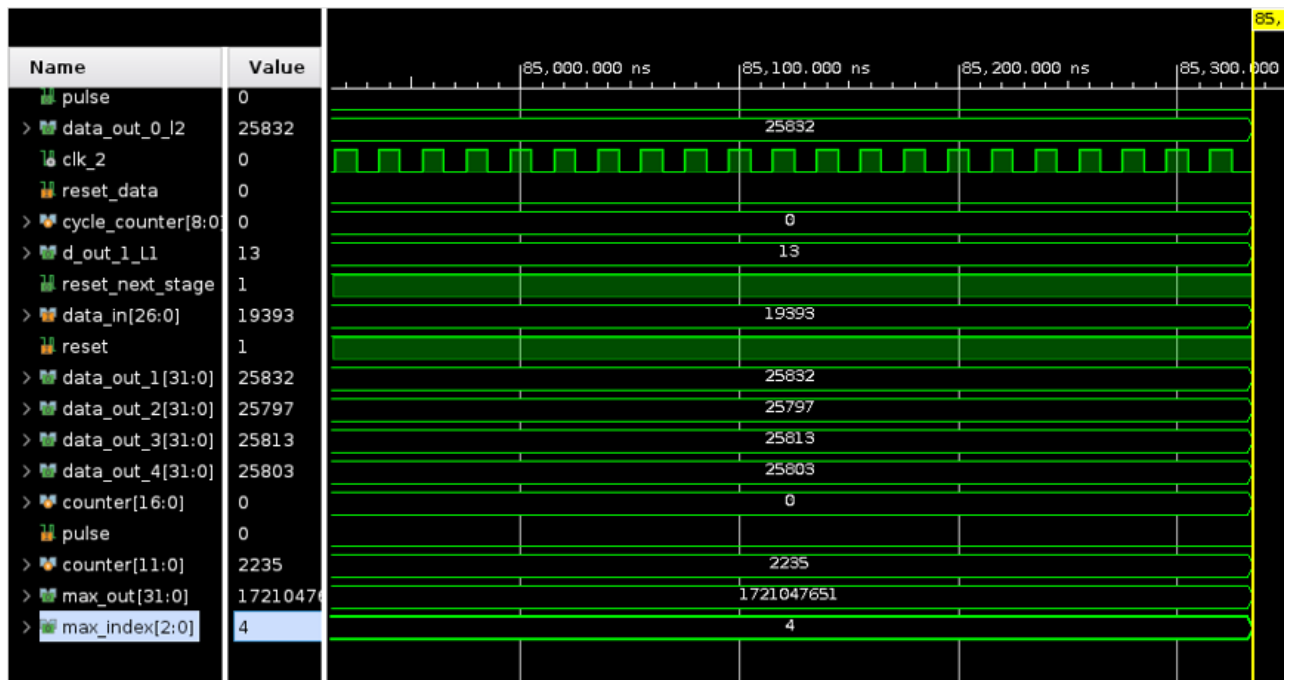


Figure 6-9: Simulation Result of CNN Model 2

Figure.6-9 shows the output of gesture recognition in Model 2 (gesture), by giving input of shape 1019 x 6 (1019 time steps and 6 channels) through BRAM.

6.3 Comparison between Resource utilisation

Table 6.1 shows the system’s hardware resource usage for both the optimized and unoptimized models of Model 1, as well as Model 2, listing LUTs, FFs, and DSPs, which helps assess the design’s complexity and scalability. In the optimized Model 1, a multiplexer is incorporated within the convolutional layer to enable the reuse of a single filter hardware block by sequentially selecting different sets of weights, significantly reducing the need for 64 parallel filter instances and thereby lowering overall hardware resource consumption while maintaining functional correctness. Model 2, which processes 2D input data using 1D convolution across each row, further optimizes hardware usage by reusing a set of 16 filter units to sequentially compute outputs for all 64 filters. This is achieved through a multiplexer-based control mechanism that dynamically switches filter weights, allowing efficient time-sharing of hardware resources. As a result, Model 2 strikes an effective balance between computational complexity and resource efficiency, making it well-suited for handling more complex input structures with minimal increase in hardware overhead.

Table 6.1: Resource Utilization

Resource	Model 1 Unop- timized (1D convolution on 1D input with no hard- ware reuse)	Model 1 Opti- mized (1D convolution on 1D input, hardware reused for different filters)	Model 2 (1D convolution on 2D in- put, hardware reused for different filters by switch- ing weights via mux)
LUTs	1,565,894	96,300	94539
FF	1,256,893	80,900	56840
DSPs	1,263	448	1280

Explanation:

In the initial implementation of **Model 1 (Unoptimized)**, the resource consumption in terms of Look-Up Tables (LUTs), Flip-Flops (FFs), and Digital Signal Processing (DSP) blocks was found to be prohibitively high. This version deployed dedicated hardware for each filter, resulting in excessive parallelism and unscalable hardware complexity. To address this limitation, we introduced a hardware-optimized variant of the model.

In the **Optimized Model 1**, the same hardware block was reused iteratively for computing outputs across all 64 filters. This significantly reduced the hardware footprint by a factor of 64, trading off throughput for resource efficiency. While this approach increased latency due to sequential execution of filter computations, it provided a practical solution with a feasible resource envelope for FPGA deployment.

Model 2, on the other hand, demonstrated improved inference accuracy during software evaluation. This was primarily due to its ability to process multi-channel temporal features, which enhanced its representational capacity. However, the increased input dimensionality led to higher resource demands. To maintain scalability, we adopted a grouped computation strategy wherein 16 filters were processed concurrently and the results were accumulated across multiple cycles. This modular instantiation enabled a controlled trade-off between computational latency and hardware utilization.

Moreover, Model 2 employed selective hardware reuse and weight multiplexing through dynamic switching mechanisms to minimize redundant resource instantiation. This strategic combination of architectural modularity and temporal reuse ensured a balanced design, optimizing both performance and efficiency for deployment in resource-constrained environments.

6.4 Comparison between Performamnce metrics

Table 6.2 outlines the key performance parameters used to assess the effectiveness and efficiency of the hardware implementation in real-time conditions. These parameters include the maximum operating frequency (Fmax), which indicates the highest clock speed at which the system can reliably function; the inference time, representing the total latency required to process a single input through the CNN pipeline; and power consumption, which reflects the energy efficiency of the design during continuous operation. Together, these metrics provide a comprehensive evaluation of the system’s real-time performance, helping to compare different architectural configurations in terms of speed, responsiveness, and energy cost—critical factors for deployment in embedded and wearable applications.

Table 6.2: Performance Metrics

Metric	Model 1 Optimized	Model 2
Fmax (MHz)	55.2	75.9
Inference Time (microseconds)	8.1	84.720
Power Consumption (W)	0.022	0.519

Chapter 7

Conclusion

This thesis presents an in-depth exploration of hardware-efficient implementations of Convolutional Neural Networks (CNNs) for gesture recognition through temporal sequences, accommodating both single and multi-channel inputs, and tailored for deployment on Field-Programmable Gate Arrays (FPGAs). The design emphasis was on optimizing hardware resource utilization while preserving computational accuracy—making the architectures well-suited for real-time, embedded, and edge AI applications.

Three architectural models were developed and analyzed, each reflecting a trade-off between performance and resource consumption. The baseline, **Model 1 (Unoptimized)**, used dedicated convolutional hardware per filter, delivering high throughput but at the cost of excessive hardware usage: over 1.5 million Look-Up Tables (LUTs), 1.2 million Flip-Flops (FFs), and 1,263 Digital Signal Processing (DSP) blocks—rendering it impractical for most FPGA targets.

To resolve this, **Model 1 (Optimized)** employed temporal hardware reuse and weight multiplexing. This approach reduced hardware usage by over 90%, bringing the footprint down to 96,300 LUTs, 80,900 FFs, and 448 DSPs. Despite a serialized execution model, performance remained competitive with

a maximum operating frequency (F_{max}) of 55.2 MHz, inference latency of 8.1 μ s, and power consumption of just 0.022 W, underscoring its efficiency for low-power, real-time deployments and but less spatial feature from single channel only accuracy is somewhere restricted to 81.36 percent only.

Model 2 advanced the architecture further to support 1D convolution across 2D multi-channel input data, enabling richer temporal-spatial feature extraction. This model maintained hardware efficiency using grouped filter computations and dynamic weight switching, consuming 94,539 LUTs, 56,840 FFs, and 1,280 DSPs. Notably, it achieved a higher F_{max} of 75.9 MHz, but incurred greater latency (84.72 μ s) and power consumption (0.519 W) due to its expanded feature capacity and processing complexity but the accuracy of this model is very high that is around 95.46 percent. These results highlight the trade-off between expressiveness and efficiency, positioning Model 2 as suitable for applications at places with less stringent power or latency constraints ,because of high accuracy because more spatial features from six input channels .

In summary, this work demonstrates that strategic architectural optimizations—including temporal reuse, modularization, and dynamic resource sharing—can drastically reduce resource consumption without severely compromising system performance. These techniques enable practical CNN deployment on FPGAs in power- and area-constrained environments such as embedded systems and wearables.

Chapter 8

Future work

While the current implementation demonstrates a functional and resource-efficient inference pipeline using fixed-point arithmetic, there are several directions for future enhancement to improve numerical precision, model performance, and real-world applicability.

Floating-Point Extension: The existing design relies on fixed-point arithmetic, which is advantageous for hardware efficiency but limited in terms of dynamic range and precision. As a future enhancement, the computational modules (adder, multiplier, activation units) will be extended to support IEEE 754 floating-point formats, such as FP16 or FP32. This will allow for more accurate computations, especially in scenarios involving high dynamic range inputs or deep model architectures. Floating-point support is expected to reduce quantization errors and saturation issues, thereby improving the overall inference accuracy.

Replacement of ReLu with tanh: The current model uses the ReLu activation function, which is computationally efficient but discards all negative activations, potentially losing useful information. In future work, the ReLU activation will be replaced with the hyperbolic tangent (**tanh**) activation function, which provides a smooth and bounded output in the range $[-1, 1]$.

This can be particularly beneficial in models where symmetric activation behavior leads to better convergence and generalization. Hardware-friendly implementations of `tanh` using Taylor series approximation or non-restoring division will be explored for efficient integration.

Real-Time Data Testing and Deployment: Currently, the system is validated using pre-collected or synthetic datasets. To evaluate its performance in practical scenarios, future efforts will involve testing the model on real-time data streams. This includes interfacing the hardware with live sensor inputs or data acquisition systems to assess inference latency, data throughput, and robustness.

Preprocessing unit We can implement EMG signal preprocessing directly in hardware, which is usually performed in software. This hardware-based preprocessing can improve real-time performance, reduce latency, and enhance overall system efficiency for edge deployments.

Appendix A

Results

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 6.841 ns	Worst Hold Slack (WHS): 0.010 ns	Worst Pulse Width Slack (WPWS): 9.457 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 253500	Total Number of Endpoints: 253500	Total Number of Endpoints: 58456

All user specified timing constraints are met.

Figure A-1: Post Implementation Timing Report(Model 2)

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 1.112 W
Design Power Budget: Not Specified
Process: typical
Power Budget Margin: N/A
Junction Temperature: 26.0°C
 Thermal Margin: 74.0°C (84.7 W)
 Ambient Temperature: 25.0 °C
 Effective θ_{JA} : 0.9°C/W
 Power supplied to off-chip devices: 0 W
 Confidence level: Low
[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

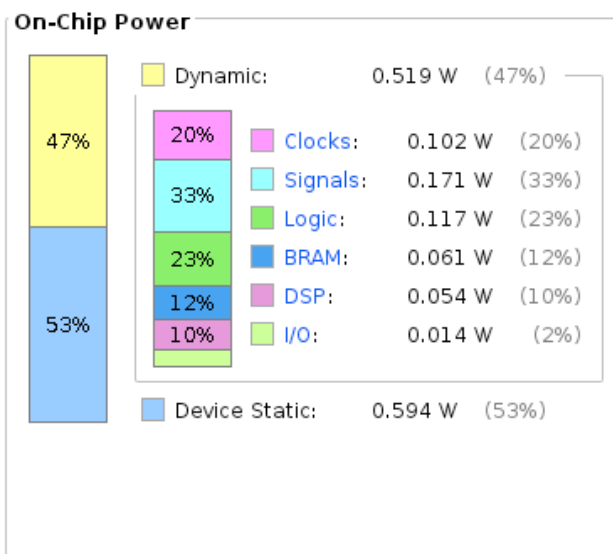


Figure A-2: Post Implementation Power report(Model 2)

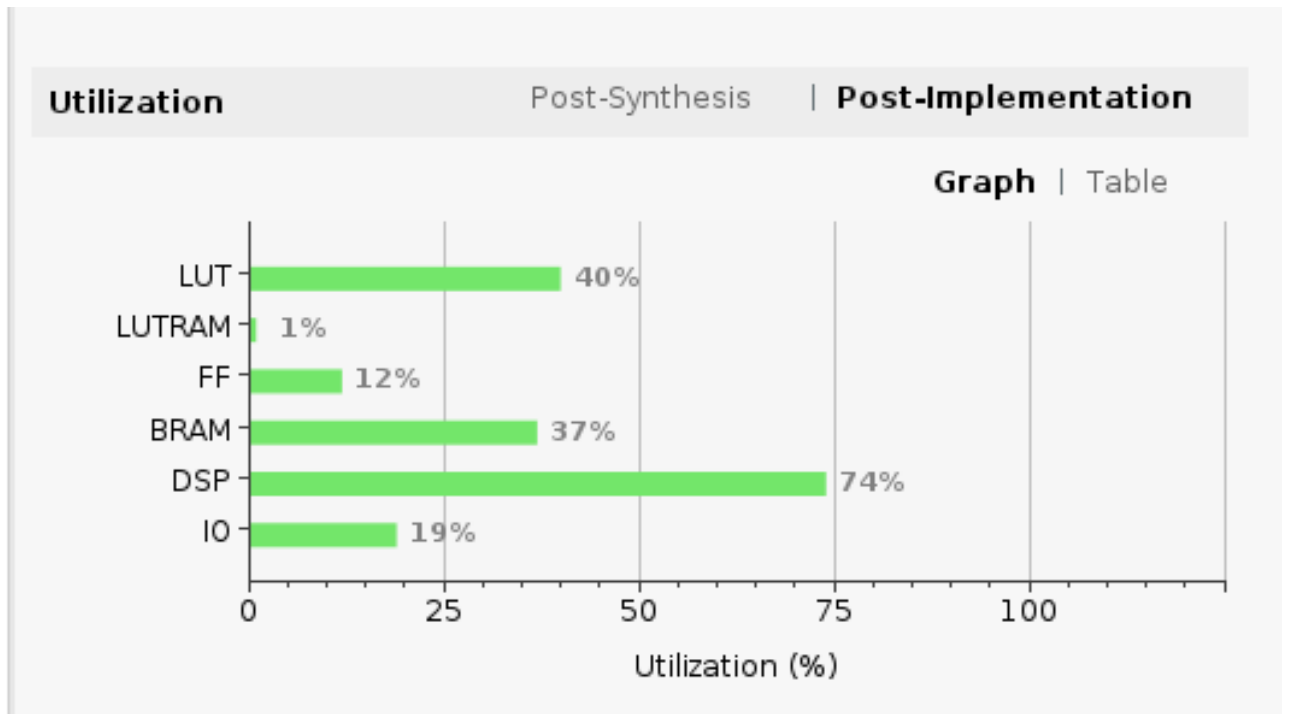


Figure A-3: Post implementation Resource utilization percentage(Model 2)

Name	CLB LUTs (230400)	CLB Registers (460800)	CARRY8 (28800)	F7 Muxes (115200)	CLB (28800)	LUT as Logic (230400)	LUT as Memory (101760)	Block RAM Tile (312)	DSPs (1728)	Bonded IOB (464)	HPIOB_M (192)	HPIOB_S (192)
top_mp5	92975	56843	6132	128	17316	92974	1	114.5	1280	86	20	20
a0 (Layer_1)	31876	14368	3580	0	6221	31876	0	17	0	0	0	0
a1 (pre_fetch)	57573	38085	2162	128	11384	57572	1	32.5	1024	0	0	0
a2 (Dense1_top_module)	1055	1138	128	0	343	1055	0	32.5	128	0	0	0
a3 (interface_Dense1_and_Dense2)	1033	2055	0	0	567	1033	0	0	0	0	0	0
a4 (Dense2_top_module)	1065	1143	128	0	344	1065	0	32.5	128	0	0	0
a5 (hard_max)	20	54	2	0	7	20	0	0	0	0	0	0

Figure A-4: Post implementation Resource utilization of individual module(Model 2)

Appendix B

Code Repository

The complete Verilog HDL code base developed for this project, including all modules for this Hardware Accelerator used in this HGR system is given below The repository can be accessed at the following link:

<https://drive.google.com/drive/folders/1pN8rHceIZcyK0sj7xnJrYVwIwWAvqU>

All files are organized by functionality and include testbenches for simulation. This resource supports reproducibility and further development of the presented work.

Bibliography

- [1] Patricio J. Cruz, Juan Pablo Vásconez, Ricardo Romero, Alex Chico, Marco E. Benalcázar, Robin Álvarez, Lorena I. Barona López, and Ángel Leonardo Valdivieso Caraguay. A deep q-network based hand gesture recognition system for control of robotic platforms. *Scientific Reports*, 13:7956, 2023.
- [2] Jakob Kaiser, Maria Magdalena Crespo-Llado, Chiara Turati, and Elena Geangu. The development of spontaneous facial responses to others' emotions in infancy: An emg study. *Scientific Reports*, 7:17500, 2017.
- [3] M.N. Mohd Nor, R. Jailani, and N.M. Tahir. Analysis of emg signals of ta and gas muscles during walking of autism spectrum disorder (asd) children. In *2016 IEEE Symposium on Computer Applications Industrial Electronics (ISCAIE)*, pages 226–230, 2016.
- [4] Zhenyu Wang. A digits-recognition convolutional neural network on fpga, 2019.
- [5] Yusen Guo, Guangyang Gou, Pan Yao, Fupeng Gao, Tianjun Ma, Jianhai Sun, Mengdi Han, Jianqun Cheng, Chunxiu Liu, Ming Zhao, and Ning Xue. Fpga-based lightweight qds-cnn system for semg gesture and force level recognition. <https://ieeexplore.ieee.org/abstract/document/10430450>, 2024.